

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
”КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО”

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

До захисту допущено:

В.о. завідувача кафедри

Артем ВОЛОКИТА

(підпис)

“ ”

2026 р.

Дипломний проєкт

на здобуття ступеня бакалавра

за освітньо-професійною програмою ”Комп’ютерні системи та мережі”
спеціальності 123 ”Комп’ютерна інженерія”

на тему: Кросплатформна клієнт-серверна система агрегації та обробки
медико-біологічних даних з використанням хмарних технологій

Виконав : студент 4 курсу, групи ІО-25
(шифр групи)

Льоскін Іван Вадимович

(прізвище, ім’я, по батькові)

(підпис)

Керівник посада, науковий ступінь, Кобилюк А. Г.

(посада, науковий ступінь, вчене звання, прізвище та ініціали)

(підпис)

Консультант (Нормоконтроль) ас. каф. ОТ Нікольський С.С.

(назва розділу) (посада, вчене звання, науковий ступінь, прізвище та ініціали)

(підпис)

Рецензент

(посада, науковий ступінь, вчене звання, прізвище та ініціали)

(підпис)

Засвідчую, що у цьому дипломному
проєкті немає запозичень з праць інших
авторів без відповідних посилань.

Студент

(підпис)

Київ — 2026 р.

Національний технічний університет України
"Київський політехнічний інститут імені Ігоря Сікорського"
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Освітньо-кваліфікаційний рівень: Бакалавр

Освітньо-професійна програма: «Комп'ютерні системи та мережі»

Спеціальність: 123 — «Комп'ютерна інженерія»

ЗАТВЕРДЖУЮ

В.о. завідувача кафедри

Артем ВОЛОКИТА

_____ (підпис)

_____ 2026 р.

_____ (число)

_____ (місяць)

ЗАВДАННЯ

на бакалаврський дипломний проєкт

Льоскіну Івану Вадимовичу

1. Тема проєкту: Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій
керівник Кобилюк Андрій Григорович, посада, науковий ступінь
затверджено наказом по університету від 5 червня 2026 року №1212-с
2. Дата здачі закінченого проєкту 5 червня 2026 року
(число, місяць, рік)
3. Вихідні дані для проєкту: технічна документація, теоретичні та статистичні дані, патенти на винахід, наукові публікації.
4. Зміст розрахунково-пояснювальної записки: опис предметної області і напрямків дослідження, аналіз та характеристика об'єкту досліджень, аналіз існуючих рішень, опис архітектури системи.

5. Перелік графічного матеріалу: принципова, структурна та функціональна схеми системи.

6. Консультанти розділів проєкту

№	Розділ	Консультант (прізвище та ініціали)	Підпис, дата	
			завдання видав	завдання прийняв
1	Нормоконтроль	Нікольський С.С.		

7. Дата видачі завдання 10 травня 2026 року
(число, місяць, рік)

Календарний план

№	Назва етапу виконання дипломного проєкту	Термін виконання	Примітка
1	Затвердження теми проєкту	06.04.2026–07.06.2026	
2	Вивчення та аналіз завдання	02.05.2026–08.05.2026	
3	Розробка архітектури та загальної структури системи	09.05.2026–15.05.2026	
4	Програмна реалізація системи	16.05.2026–29.05.2026	
5	Оформлення пояснювальної записки	30.05.2026–07.06.2026	
6	Передзахист	06.06.2026	
7	Захист	20.06.2026	

Виконавець _____ Льоскін Іван Вадимович
(підпис)

Керівник _____ Кобилюк Андрій Григорович
(підпис)

АНОТАЦІЯ

Пояснювальна записка містить 69 сторінок, 10 рисунків, 2 таблиці, 38 джерел.

У даній дипломній роботі проведено огляд та аналіз існуючих мобільних застосунків для відстеження розвитку немовляти, досліджено їх переваги та недоліки. На основі аналізу предметної області та потреб користувачів сформульовано вимоги до системи, обрано технологічний стек і спроектовано архітектуру кросплатформного мобільного застосунку. Розроблено реляційну схему бази даних із захистом на рівні рядків (Row-Level Security), що забезпечує ізоляцію даних між користувачами, та визначено ключові модулі системи.

В результаті розроблено кросплатформний мобільний застосунок для платформ iOS та Android, що поєднує щоденник активностей немовляти, моніторинг антропометричних показників відповідно до стандартів ВООЗ, управління вакцинаціями та досягненнями, а також ІІІ-асистента на основі технології Retrieval-Augmented Generation. Застосунок реалізовано з використанням React Native та Expo; серверну частину побудовано на Supabase із PostgreSQL, pgvector та Row-Level Security. ІІІ-асистента інтегровано через Groq API з векторним пошуком у базі педіатричних знань. Розгортання здійснено на хмарній інфраструктурі з підтримкою синхронізації між пристроями в реальному часі. Проведено тестування системи на рівнях юніт-тестів, тестів бази даних і наскрізних тестів.

Ключові слова: мобільний застосунок, React Native, Expo, Supabase, pgvector, RAG, ІІІ-асистент, відстеження розвитку немовляти, ВООЗ.

ANNOTATION

The explanatory note contains 69 pages, 10 figures, 2 tables, 38 references.

In this bachelor's thesis, a review and analysis of existing mobile applications for infant development tracking was conducted, and their advantages and shortcomings were examined. Based on the subject domain analysis and user needs, system requirements were formulated, a technology stack was selected, and the architecture of a cross-platform mobile application was designed. A relational database schema with Row-Level Security was developed to ensure data isolation between users, and the key system modules were defined.

As a result, a cross-platform mobile application for iOS and Android platforms was developed, combining an infant activity journal, monitoring of anthropometric indicators in accordance with WHO standards, management of vaccinations and developmental milestones, and an AI assistant based on Retrieval-Augmented Generation technology. The application was implemented using React Native and Expo; the backend was built on Supabase with PostgreSQL, pgvector and Row-Level Security. The AI assistant was integrated via the Groq API with vector search in a paediatric knowledge base. The system was deployed on cloud infrastructure with support for real-time synchronisation between devices. System testing was conducted at unit, database and end-to-end levels.

Keywords: mobile application, React Native, Expo, Supabase, pgvector, RAG, AI assistant, infant development tracking, WHO.

ТЕХНІЧНЕ ЗАВДАННЯ ДО ДИПЛОМНОГО ПРОЄКТУ

на тему: *«Кросплатформна клієнт-серверна система агрегації та обробки
медико-біологічних даних з використанням хмарних технологій»*

Київ – 2026

ЗМІСТ

1 НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ.....	2
2 ПІДСТАВИ ДЛЯ РОЗРОБКИ	2
3 МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ	2
4 ДЖЕРЕЛА РОЗРОБКИ.....	3
5 ТЕХНІЧНІ ВИМОГИ	3
5.1. Вимоги до розробленого продукту	3
5.2. Вимоги до програмного забезпечення	4
5.3. Вимоги до апаратної частини.....	4
6 ЕТАПИ РОЗРОБКИ.....	4

					<i>ІАЛЦ.467200.002 ТЗ</i>			
<i>Зм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Підп.</i>	<i>Дата</i>				
<i>Розробив</i>	<i>Льоскін І. В.</i>				<i>Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій Технічне завдання</i>	<i>Лит.</i>	<i>Аркуш</i>	<i>Аркушів</i>
<i>Перевірив</i>	<i>Кобилюк А. Г.</i>						1	4
<i>Н. контр.</i>	<i>Нікольський С.С.</i>					<i>НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ ІО-25</i>		
<i>Затвердив</i>	<i>Волокита А. М.</i>							

1 НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ

Це технічне завдання стосується розробки кросплатформного мобільного застосунку для відстеження розвитку немовляти. Застосунок призначений для батьків та опікунів, які потребують зручного інструменту для моніторингу активностей дитини, контролю антропометричних показників відповідно до стандартів ВООЗ, управління вакцинаціями та досягненнями, а також отримання персоналізованих педіатричних рекомендацій. Система розрахована на використання в побутових умовах на мобільних пристроях під управлінням iOS та Android.

2 ПІДСТАВИ ДЛЯ РОЗРОБКИ

Підставою для розробки цього програмного продукту є виконання вимог кваліфікаційно-освітнього рівня «бакалавр» зі спеціальності «Комп'ютерна інженерія», затвердженого відповідними освітніми та науковими установами Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

3 МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ

Метою розробки є створення зручного та функціонального кросплатформного мобільного застосунку, який допоможе батькам систематично відстежувати розвиток немовляти в перші роки життя.

Застосунок призначений для ведення щоденника активностей дитини (годування, сон, підгузки), моніторингу росту та ваги з порівнянням до стандартів ВООЗ, а також відстеження вакцинацій і вікових досягнень. Система забезпечує синхронізацію даних між пристроями в реальному часі, що дозволяє обом батькам мати актуальну інформацію одночасно.

					ІА/Ц.467200.002 ТЗ	Аркуш
						2
Зм.	Лист	№ докум.	Підп.	Дата		

4 ДЖЕРЕЛА РОЗРОБКИ

Джерелами розробки даного дипломного проєкту є офіційна технічна документація до використовуваних технологій (React Native, Expo, Supabase), стандарти ВООЗ щодо показників росту та розвитку дітей, наукові публікації у сфері мобільної розробки, а також статті та матеріали в мережі Інтернет за тематикою проєкту.

5 ТЕХНІЧНІ ВИМОГИ

5.1. Вимоги до розробленого продукту

Розроблена система повинна задовольняти наступні вимоги:

- кросплатформна підтримка iOS 16.0 і новіших та Android 10 (API 29) і новіших версій;
- автентифікація користувачів та ізоляція даних на рівні рядків бази даних (Row-Level Security);
- синхронізація даних між пристроями в реальному часі через Supabase Realtime;
- відображення графіків зростання та порівняння з еталонними кривими ВООЗ;
- підтримка кількох профілів дітей в рамках одного облікового запису;
- зрозумілий та інтуїтивний інтерфейс, що не вимагає попередньої технічної підготовки.

					ІАЛЦ.467200.002 ТЗ	Аркуш
						3
Зм.	Лист	№ докум.	Підп.	Дата		

5.2. Вимоги до програмного забезпечення

- операційна система: iOS 16.0+ або Android 10+;
- активне підключення до мережі Інтернет для синхронізації даних між пристроями;
- для розробки: Node.js 18+, Expo CLI, Git.

5.3. Вимоги до апаратної частини

- смартфон з оперативною пам'яттю не менше 3 ГБ;
- наявність підключення до мережі Інтернет (Wi-Fi або мобільний інтернет).

6 ЕТАПИ РОЗРОБКИ

Назва етапу виконання	Термін виконання
Затвердження теми роботи	06.04.2026–07.06.2026
Вивчення та аналіз завдання	02.05.2026–08.05.2026
Розробка архітектури та загальної структури системи	09.05.2026–15.05.2026
Розробка структур окремих частин системи	09.05.2026–15.05.2026
Програмна реалізація системи	16.05.2026–22.05.2026
Виправлення помилок	23.05.2026–29.05.2026
Оформлення пояснювальної записки	30.05.2026–07.06.2026

ПОЯСНЮВАЛЬНА ЗАПИСКА ДО ДИПЛОМНОГО ПРОЄКТУ

на тему: *«Кросплатформна клієнт-серверна система агрегації та обробки
медико-біологічних даних з використанням хмарних технологій»*

Київ – 2026

ЗМІСТ

ПЕРЕЛІК СКОРОЧЕНЬ.....	3
ВСТУП	4
РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ СИСТЕМ МОНІТОРИНГУ РОЗВИТКУ НЕМОВЛЯТИ ТА ОГЛЯД ІСНУЮЧИХ РІШЕНЬ	6
1.1 Актуальність та доцільність розроблення.....	6
1.2 Огляд та аналіз існуючих рішень.....	7
1.2.1 Sprout Baby Tracker	7
1.2.2 Huckleberry	8
1.2.3 Glow Baby.....	9
1.3 Аналіз функціональності та вимог до системи	10
1.3.1 Порівняльна характеристика розглянутих рішень.....	10
1.3.2 Вимоги до реалізації застосунку	12
ВИСНОВОК ДО РОЗДІЛУ 1	14
РОЗДІЛ 2. ОГЛЯД ТА ВИБІР ТЕХНОЛОГІЙ ДЛЯ ПРОЕКТУВАННЯ СИСТЕМИ.....	15
2.1 Загальні принципи побудови кроссплатформних мобільних застосунків.....	15
2.1.1 Клієнт-серверна архітектура мобільних застосунків.....	15
2.1.2 Основні компоненти мобільного застосунку.....	16
2.2 Огляд технологій для реалізації застосунку.....	18
2.2.1 Технології клієнтської частини.....	18
2.2.2 Хмарна платформа та серверна частина	19
2.2.3 Характеристика системи управління базами даних.....	19
2.2.4 Управління станом, локалізація та допоміжні бібліотеки ...	20
2.3 Механізми автентифікації та захисту даних.....	20
2.3.1 Row-Level Security та ізоляція даних користувачів	21
2.3.2 Безпека мобільного застосунку	21
2.4 Стандарти ВООЗ та педіатричний моніторинг	22
2.5 Ключові алгоритми обробки медико-біологічних даних	23
2.5.1 Алгоритм ІІІ-асистента на основі RAG	23
2.5.2 Алгоритм перцентильної класифікації антропометричних показників	25
ВИСНОВОК ДО РОЗДІЛУ 2	26

					<i>ІАЛЦ.467200.003 ПЗ</i>			
<i>Зм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Підп.</i>	<i>Дата</i>				
<i>Розробив</i>	<i>Льоскін І. В.</i>				Кроссплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій Пояснювальна записка	<i>Лит.</i>	<i>Аркуш</i>	<i>Аркушів</i>
<i>Перевірів</i>	<i>Кобилюк А. Г.</i>						1	72
<i>Н. контр.</i>	<i>Нікольський С.С.</i>					НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ ІО-25		
<i>Затвердив</i>	<i>Волокита А. М.</i>							

РОЗДІЛ 3. ПРОЕКТУВАННЯ ТА РЕАЛІЗАЦІЯ КРОСПЛАТФОРМНОЇ СИСТЕМИ АГРЕГАЦІЇ МЕДИКО-БІОЛОГІЧНИХ ДАНИХ.....	28
3.1 Архітектура системи.....	28
3.1.1 Загальна архітектурна схема застосунку.....	28
3.1.2 Архітектурні особливості клієнтської частини	29
3.1.3 Архітектура хмарної серверної частини	31
3.1.4 Архітектура бази даних.....	32
3.2 Реалізація серверної частини	34
3.2.1 Проєктування та реалізація бази даних.....	34
3.2.2 Реалізація Row-Level Security.....	35
3.3 Реалізація клієнтської частини	37
3.3.1 Організація структури модулів	37
3.3.2 Моніторинг помилок та безпека середовища виконання	39
3.3.3 Навігаційна структура та управління станом	40
3.3.4 Зберігання файлів та синхронізація в реальному часі	42
3.3.5 Реалізація клієнтської автентифікації	43
3.3.6 Реалізація ключових екранів та компонентів	45
3.3.7 Робота з датами та часом.....	49
3.3.8 Реалізація алгоритмів обробки медичних даних	49
ВИСНОВОК ДО РОЗДІЛУ 3	52
РОЗДІЛ 4. ТЕСТУВАННЯ ТА РОЗГОРТАННЯ СИСТЕМИ АГРЕГАЦІЇ МЕДИКО-БІОЛОГІЧНИХ ДАНИХ.....	53
4.1 Демонстрація роботи та функціональне тестування застосунку... 53	
4.1.1 Реєстрація та автентифікація користувача	53
4.1.2 Додавання та відстеження активностей	54
4.1.3 Моніторинг антропометричних показників	57
4.1.4 Аналітика активностей.....	58
4.1.5 Параметри профілю та персоналізація інтерфейсу	59
4.2 Тестування ключових функцій системи.....	60
4.2.1 Модульне тестування бізнес-логіки.....	60
4.2.2 Перевірка безпеки та ізоляції даних	62
4.2.3 Тестування синхронізації між пристроями	63
4.3 Аналіз результатів тестування.....	64
4.4 Розгортання бекенду та публікація застосунку	65
ВИСНОВОК ДО РОЗДІЛУ 4	67
ВИСНОВКИ.....	68
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	69

ПЕРЕЛІК СКОРОЧЕНЬ

API	(Application Programming Interface) прикладний програмний інтерфейс
ER	(Entity-Relationship) сутність-зв'язок
HTTPS	(HyperText Transfer Protocol Secure) захищений протокол передачі гіпертексту
JWT	(JSON Web Token) веб-токен JSON
OAuth	(Open Authorization) відкритий протокол авторизації
REST	(Representational State Transfer) передача репрезентативного стану
RLS	(Row-Level Security) безпека на рівні рядків
SDK	(Software Development Kit) набір інструментів розробника
SQL	(Structured Query Language) мова структурованих запитів
UI	(User Interface) інтерфейс користувача
URL	(Uniform Resource Locator) уніфікований локатор ресурсу
БД	база даних
ВООЗ	Всесвітня організація охорони здоров'я
СУБД	система управління базами даних
ШІ	штучний інтелект

ВСТУП

У сучасному суспільстві батьки та опікуни щодня стикаються з необхідністю систематичного відстеження медико-біологічних показників новонародженої дитини: режиму годування та сну, динаміки маси тіла й зросту, графіка вакцинацій і вікових досягнень. ВООЗ наголошує, що регулярний моніторинг фізичного розвитку протягом перших двох років життя є ключовим інструментом раннього виявлення відхилень і своєчасного медичного втручання. Попри широкий вибір мобільних застосунків для моніторингу немовляти, жоден із них не поєднує в собі повноцінного антропометричного моніторингу відповідно до стандартів ВООЗ, синхронізації між пристроями в реальному часі. Це зумовлює актуальність розроблення комплексної кросплатформної системи, яка усуне зазначені недоліки.

Метою дипломного проєкту є розроблення кросплатформної клієнт-серверної системи агрегації та обробки медико-біологічних даних з використанням хмарних технологій — мобільного застосунку для відстеження розвитку немовляти на платформах iOS та Android. Об'єктом дослідження є процеси агрегації, зберігання та аналізу медико-біологічних даних про розвиток немовляти. Предметом дослідження є методи та інструменти розроблення кросплатформних мобільних застосунків із застосуванням хмарних технологій.

У процесі виконання роботи буде проведено аналіз предметної області та огляд існуючих аналогів, сформульовано вимоги до системи. Також буде здійснено огляд та обґрунтування вибору технологічного стеку: React Native та Expo для клієнтської частини, Supabase з PostgreSQL для хмарного бекенду. Наступні етапи передбачають проєктування реляційної схеми бази даних із захистом на рівні рядків (RLS), реалізацію всіх модулів застосунку,

а також тестування системи на рівнях юніт-тестів та функціонального тестування.

Розроблений застосунок є актуальним практичним рішенням для широкого кола користувачів — від молодих батьків, які вперше стикаються з необхідністю ведення медичних записів дитини, до сімей, де обидва батьки потребують спільного доступу до актуальних даних у режимі реального часу. Застосунок розгорнуто на хмарній інфраструктурі та готовий до використання на обох платформах.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						5
Зм.	Лист	№ докум.	Підп.	Дата		

РОЗДІЛ 1

АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ СИСТЕМ МОНІТОРИНГУ РОЗВИТКУ НЕМОВЛЯТИ ТА ОГЛЯД ІСНУЮЧИХ РІШЕНЬ

1.1 Актуальність та доцільність розроблення

За даними ВООЗ, перші два роки життя дитини є найбільш інтенсивним періодом фізичного розвитку. Регулярний моніторинг маси тіла, зросту та окружності голови дозволяє педіатрам вчасно виявляти відхилення від вікових норм і призначати корекційні заходи на ранніх стадіях. Паралельно батьки щодня фіксують режим годування та сну, стан здоров'я, вакцинації та вікові досягнення — дані, що формують повноцінну картину розвитку дитини і є незамінними під час лікарських консультацій.

Попри важливість систематичного ведення таких записів, більшість батьків і сьогодні покладаються на паперові щоденники або стандартні нотатки смартфона. Такий підхід має суттєві обмеження: дані легко втрачаються, їх неможливо автоматично перетворити на динамічні графіки, а ведення записів одночасно двома батьками з різних пристроїв призводить до неузгодженостей і втрати актуальності даних. Крім того, паперові записи не дозволяють зіставити показники дитини зі стандартними перцентильними кривими, що унеможливорює самостійну оцінку динаміки розвитку між лікарськими візитами.

У відповідь на ці потреби ринок мобільних застосунків для батьків активно розвивається. Проте, як показує аналіз наявних рішень, переважна більшість із них охоплює лише окремі аспекти моніторингу і не забезпечує комплексного підходу. Жоден із популярних застосунків не поєднує

повноцінного антропометричного моніторингу відповідно до стандартів ВООЗ, синхронізації між пристроями в реальному часі — що визначає практичну актуальність розроблення такої системи.

1.2 Огляд та аналіз існуючих рішень

На ринку мобільних застосунків для батьків існує декілька поширених рішень для відстеження розвитку немовляти. Їх функціональність варіюється від базового журналювання активностей до комплексного відстеження сну з аналітикою. Було розглянуто три найбільш популярних застосунки: Sprout Baby Tracker, Huckleberry та Glow Baby. Аналіз цих рішень дозволив виявити їхні ключові переваги та недоліки, а також визначити напрями для вдосконалення у розроблюваній системі.

1.2.1 Sprout Baby Tracker

Sprout Baby Tracker — один із найстаріших і найвідоміших застосунків для відстеження немовляти, доступний на платформах iOS та Android. Застосунок орієнтований на ведення повноцінного журналу активностей дитини з перших днів життя і дозволяє фіксувати годування (грудне молоко та суміш), епізоди сну, зміни підгузників, вимірювання маси тіла та зросту, а також переглядати хронологію подій у вигляді стрічки. Передбачена підтримка декількох профілів дітей в одному обліковому записі, що зручно для сімей із двома і більше дітьми.

До переваг Sprout Baby Tracker належать зрілий і зручний інтерфейс, широка функціональність журналювання та надійна робота без підключення до мережі. Графіки зростання дозволяють відстежувати динаміку антропометричних показників, однак порівняння з перцентильними кривими ВООЗ безпосередньо в застосунку відсутнє. Синхронізація між пристроями реалізована через хмарне резервне копіювання і не відбувається

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		7

в реальному часі, що ускладнює спільне ведення записів двома батьками.

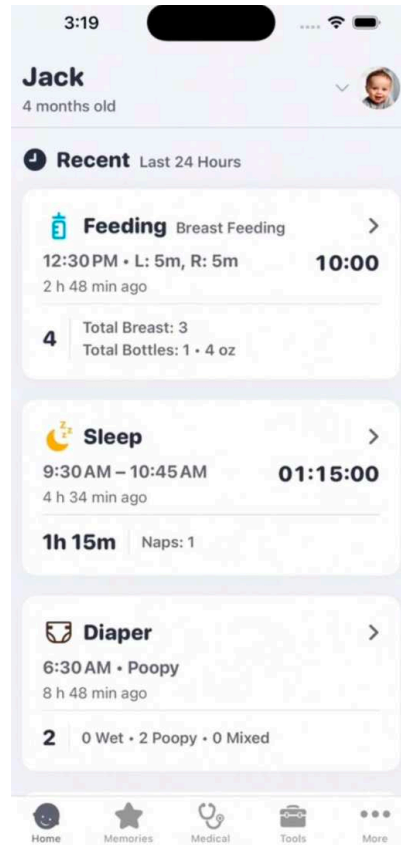


Рисунок 1.1 – Інтерфейс застосунку Sprout Baby Tracker

1.2.2 Huckleberry

Huckleberry — застосунок, орієнтований передусім на аналіз сну немовляти. Його ключова функція — алгоритм SweetSpot, який на основі накопичених даних про денний сон і нічний відпочинок пропонує оптимальний час для вкладання дитини. Застосунок збирає дані про тривалість і якість сну, фіксує годування та дозволяє вести загальні нотатки. Преміальна версія надає доступ до персоналізованих консультацій із сертифікованими фахівцями з дитячого сну.

Перевагою Huckleberry є глибока аналітика сну та наявність консультаційного сервісу, що виділяє його серед аналогів. Водночас застосунок має суттєві обмеження: повноцінний модуль антропометричного моніторингу з перцентильними кривими BOOЗ відсутній, функціональність

безкоштовної версії значно обмежена, а покрокового журналу активностей, який охоплював би всі аспекти догляду за немовлям, немає.

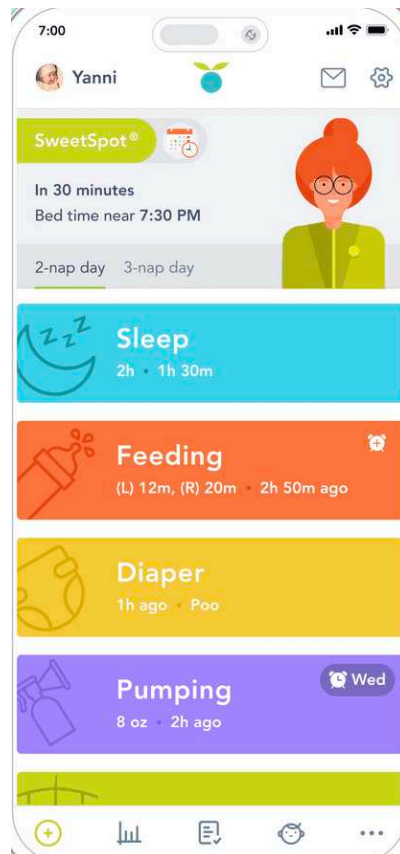


Рисунок 1.2 – Інтерфейс застосунку Huckleberry

1.2.3 Glow Baby

Glow Baby — комплексний трекер розвитку дитини від компанії Glow Inc., доступний на iOS та Android. Застосунок охоплює журнал активностей (годування, сон, підгузники, купання), відстеження вікових досягнень за категоріями (моторика, мова, соціальний розвиток), базові графіки зростання та широку базу статей про догляд за немовлятами. Передбачена функція спільного доступу, що дозволяє обом батькам переглядати та додавати записи.

Glow Baby вирізняється широтою контенту, системою вікових підказок і наявністю спільного доступу для батьків. Проте порівняння антропометричних показників із перцентильними кривими BOOЗ

					ІАЛЦ.467200.003 ПЗ	Аркуш
						9
Зм.	Лист	№ докум.	Підп.	Дата		

реалізоване спрощено і не відображає повний набір стандартів. Статті носять загальний інформаційний характер без прив’язки до конкретних даних дитини. Синхронізація між пристроями в режимі реального часу також не забезпечена.

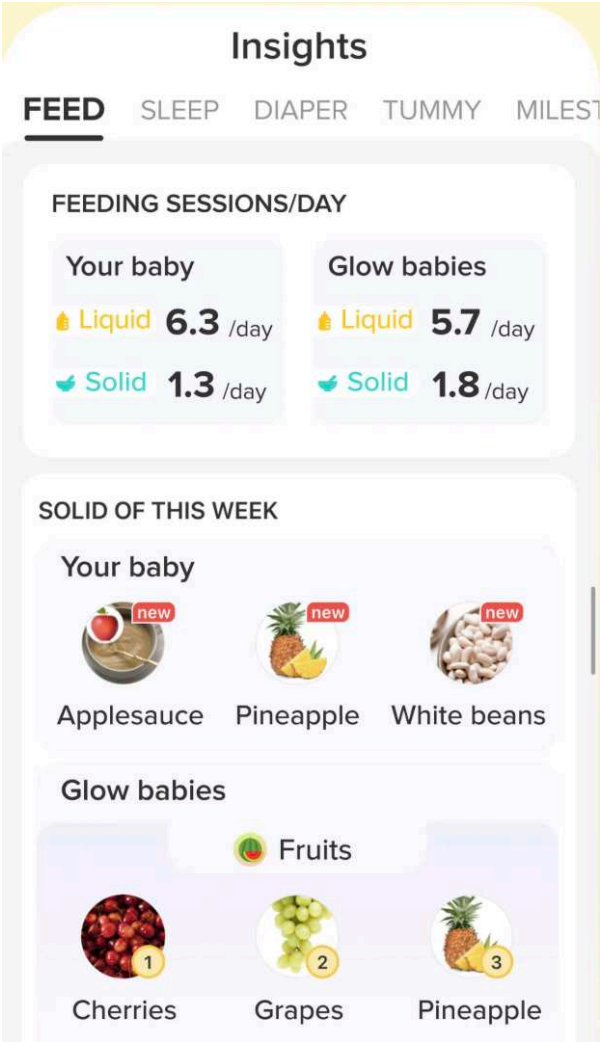


Рисунок 1.3 – Інтерфейс застосунку Glow Baby

1.3 Аналіз функціональності та вимог до системи

1.3.1 Порівняльна характеристика розглянутих рішень

На основі огляду трьох застосунків проведено порівняльний аналіз їхніх ключових переваг і недоліків. Результати наведено в таблиці 1.1.

Таблиця 1.1 – Порівняльна характеристика існуючих рішень

Застосунок	Переваги	Недоліки
Sprout Baby Tracker	1. Широкий журнал активностей 2. Підтримка декількох профілів дітей 3. Надійна робота без мережі	1. Відсутність кривих ВООЗ 2. Синхронізація не в реальному часі
Huckleberry	1. Глибока аналітика сну 2. Алгоритм SweetSpot 3. Чат з консультантом зі сну	1. Відсутність антропометричного модуля 2. Обмежений безкоштовний функціонал
Glow Baby	1. Широкий контент та вікові підказки 2. Спільний доступ для батьків 3. База статей про догляд	1. Спрощені графіки без кривих ВООЗ 2. Загальні статті без прив'язки до даних

Проведений порівняльний аналіз демонструє, що кожна з розглянутих платформ має свої сильні сторони, але водночас і певні обмеження. Sprout Baby Tracker вирізняється зрілим інтерфейсом та широким журналом активностей, але не має підтримки кривих ВООЗ та синхронізації в реальному часі. Huckleberry пропонує унікальну аналітику сну, проте є вузькоспеціалізованим і не забезпечує комплексного моніторингу. Glow Baby охоплює найширший спектр функцій серед аналогів, однак поступається у точності антропометричних графіків і не має персоналізованого

III-асистента. Таким чином, жоден із розглянутих застосунків не поєднує повноцінного антропометричного моніторингу відповідно до стандартів ВООЗ та синхронізації між пристроями в реальному часі, що обґрунтовує доцільність розроблення нового рішення.

Для наочного позиціонування розроблюваної системи відносно існуючих аналогів складено функціональну матрицю порівняння, наведену в таблиці 1.2.

Таблиця 1.2 – Функціональна матриця порівняння застосунків

Критерій	Sprout Baby Tracker	Huckleberry	Glow Baby	Розроблювана система
Журнал активностей	Так	Частково	Так	Так
Антропометрія з кривими ВООЗ	Ні	Ні	Частково	Так
Синхронізація в реальному часі	Ні	Ні	Ні	Так
Аналітика режиму сну	Ні	Так	Ні	Частково
Вакцинації та вікові досягнення	Частково	Ні	Частково	Так
Спільний доступ опікунів	Частково	Ні	Так	Так

1.3.2 Вимоги до реалізації застосунку

З урахуванням аналізу існуючих рішень та потреб користувачів сформульовано функціональні та нефункціональні вимоги до розроблюваної системи. Вимоги формувалися з прагненням усунути виявлені недоліки аналогів і забезпечити комплексний інструмент для відстеження розвитку немовляти.

Функціональні вимоги:

- 1) Реєстрація та автентифікація користувача через email і пароль із

захищеним зберіганням сесії.

- 2) Підтримка декількох профілів дітей та моделі опікунів: можливість запросити іншого користувача для спільного ведення записів.
- 3) Ведення журналу активностей: годування (грудне молоко, суміш, прикорм), сон, підгузники, купання, температура, ліки.
- 4) Додавання антропометричних вимірювань (маса тіла, зріст, окружність голови) та їх відображення на інтерактивних графіках із перцентильними кривими ВООЗ.
- 5) Управління графіком вакцинацій із відміткою виконаних щеплень.
- 6) Відстеження вікових досягнень дитини з відміткою дати виконання.
- 7) Синхронізація даних між пристроями в реальному часі через механізм підписки на зміни БД.

Нефункціональні вимоги:

- 1) Кросплатформна підтримка iOS 16.0+ та Android 10+ із нативним відображенням компонентів інтерфейсу.
- 2) Час відгуку API не більше 500 мс для основних операцій при стабільному мережевому з'єднанні.
- 3) Ізоляція даних між користувачами на рівні СУБД засобами RLS, незалежно від логіки клієнтського застосунку.
- 4) Масштабованість хмарної інфраструктури для обслуговування зростаючої кількості користувачів без зміни архітектури.
- 5) Захист застосунку від несанкціонованої модифікації та запуску на скомпрометованих пристроях.

Визначені функціональні та нефункціональні вимоги є основою для проєктування архітектури системи та вибору технологічного стеку в наступних розділах.

ВИСНОВОК ДО РОЗДІЛУ 1

У першому розділі проведено аналіз предметної області — системи агрегації та обробки медико-біологічних даних немовляти в мобільному застосунку. Обґрунтовано актуальність розроблення: регулярний педіатричний моніторинг є критично важливим у перші два роки життя дитини, проте більшість батьків не має зручного цифрового інструменту для його ведення з підтримкою синхронізації між пристроями в реальному часі.

Здійснено огляд трьох популярних аналогів — Sprout Baby Tracker, Huckleberry та Glow Baby. Встановлено, що кожен із них орієнтований на певний аспект моніторингу: Sprout Baby Tracker охоплює широкий журнал активностей, Huckleberry спеціалізується на аналізі сну, а Glow Baby пропонує найбільший обсяг контенту серед аналогів. Водночас жоден із розглянутих застосунків не поєднує повноцінного антропометричного моніторингу з кривими ВООЗ та синхронізації між пристроями в реальному часі.

На основі порівняльного аналізу сформульовано сім функціональних і п'ять нефункціональних вимог до розроблюваної системи. Функціональні вимоги охоплюють автентифікацію, модель опікунів, журнал активностей, антропометричний моніторинг із кривими ВООЗ, вакцинації, досягнення та синхронізацію в реальному часі. Нефункціональні вимоги регламентують продуктивність, безпеку та масштабованість системи.

Визначені вимоги покладено в основу вибору технологічного стеку та проєктування архітектури системи, які розглядаються в наступних розділах.

РОЗДІЛ 2

ОГЛЯД ТА ВИБІР ТЕХНОЛОГІЙ ДЛЯ ПРОЕКТУВАННЯ СИСТЕМИ

2.1 Загальні принципи побудови кросплатформних мобільних застосунків

2.1.1 Клієнт-серверна архітектура мобільних застосунків

Більшість сучасних мобільних застосунків побудовано за клієнт-серверною архітектурою: клієнтська частина (мобільний застосунок) відповідає за відображення даних і взаємодію з користувачем, тоді як серверна частина забезпечує зберігання, обробку та ізоляцію даних. Такий поділ дозволяє масштабувати компоненти системи незалежно: бекенд може обслуговувати одночасно iOS- та Android-клієнтів без дублювання бізнес-логіки.

У контексті розроблення мобільних клієнтів виокремлюють три основні підходи, що відрізняються балансом між продуктивністю, витратами на розробку та доступом до платформних можливостей.

Нативна розробка передбачає окремі кодові бази для iOS (Swift/Objective-C) та Android (Kotlin/Java). Цей підхід забезпечує найвищу продуктивність, повний доступ до платформних API та найкраще відчуття платформи, однак вимагає підтримки двох незалежних кодових баз, що подвоює витрати на розробку та тестування.

Гібридний підхід базується на вебтехнологіях (HTML/CSS/JavaScript), загорнутих у нативну оболонку за допомогою таких фреймворків, як Ionic або Capacitor. Спільна кодова база суттєво знижує витрати, проте застосунок виконується у WebView, що призводить до помітних затримок

					ІАЛЦ.467200.003 ПЗ	Аркуш
						15
Зм.	Лист	№ докум.	Підп.	Дата		

при рендерингу та нестандартної поведінки нативних елементів UI.

Кросплатформна нативна розробка поєднує переваги обох підходів: єдина кодова база транспілюється або компілюється у нативний код для кожної платформи. Серед представників цього підходу виокремлюють React Native від Meta та Flutter від Google. React Native відображає компоненти через нативний міст платформи, тоді як Flutter використовує власний рушій рендерингу Skia/Impeller. Обидва інструменти дозволяють досягти продуктивності, близької до нативної, при значно менших витратах на розробку.

Для розроблюваного застосунку обрано кросплатформну нативну розробку на базі React Native. Ключовими факторами вибору є зрілість екосистеми JavaScript/TypeScript, пряма інтеграція з Supabase SDK та Expo, а також можливість повторного використання вебкомпонентів. На відміну від Flutter, React Native не потребує вивчення мови Dart і органічно вписується в існуючу екосистему npm-пакетів.

2.1.2 Основні компоненти мобільного застосунку

Архітектура розроблюваного застосунку організована у чотири логічних шари з односпрямованою залежністю: вищий шар може звертатись до нижчого, але не навпаки, що запобігає циклічним залежностям і спрощує тестування.

Найближчим до користувача є шар представлення (*Presentation layer*), що містить компоненти UI, екрани та навігаційну структуру. У застосунку він реалізований на базі Expo Router із файловою маршрутизацією: кожен файл у директорії `app/` автоматично відповідає окремому маршруту, а вкладеність директорій визначає ієрархію навігації. Компоненти цього шару отримують дані виключно через пропси або React-хуки і ніколи не звертаються до API напряму.

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		16

Під ним розташований шар бізнес-логіки (*Business logic layer*), відповідальний за трансформацію даних, валідацію та управління станом застосунку. Він реалізований через кастомні React-хуки та глобальне сховище Zustand, у якому зберігаються поточний профіль дитини і стан автентифікаційної сесії. Цей шар виступає посередником між представленням і даними: він формує запити до API та передає готові до відображення об'єкти вгору по стеку.

Шар даних (*Data layer*) забезпечує взаємодію з хмарним API та управління локальним кешем. Він реалізований через клієнт Supabase у поєднанні з TanStack Query, що автоматично кешує відповіді сервера, виконує фонове оновлення при поверненні застосунку на передній план і повторює невдалі запити після відновлення мережевого з'єднання.

Основу стеку складає хмарний бекенд (*Backend layer*) — PostgreSQL з увімкненим RLS, автентифікаційний сервіс Supabase Auth, Realtime-підписки для синхронізації між пристроями. Бекенд не передбачає окремого серверного застосунку: бізнес-правила інкапсульовано безпосередньо у SQL-функції та тригери бази даних, що скорочує кількість рухомих частин системи.

Взаємодію між шарами схематично зображено на рис. 2.1: кожен вищий шар залежить від нижчого, але не навпаки, що унеможлиблює циклічні залежності.

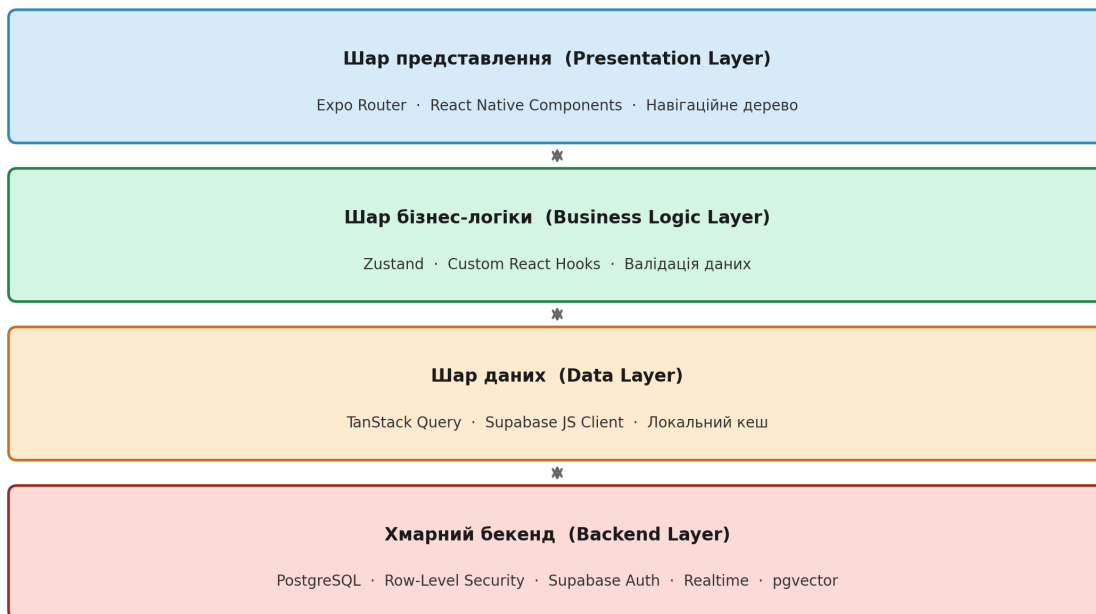


Рисунок 2.1 – Чотиришарова архітектура мобільного застосунку

2.2 Огляд технологій для реалізації застосунку

2.2.1 Технології клієнтської частини

React Native — фреймворк від компанії Meta для розроблення нативних мобільних застосунків із використанням JavaScript та TypeScript. На відміну від гібридних рішень, компоненти React Native транслюються безпосередньо у нативні елементи UI кожної платформи без проміжного WebView, що забезпечує продуктивність, порівнянну з нативними застосунками. Єдина кодова база для iOS та Android у поєднанні з великою екосистемою npm-пакетів і підтримкою гарячого оновлення (Fast Refresh) суттєво прискорює цикл розробки.

Ехро є офіційно рекомендованою надбудовою над React Native, що надає уніфікований SDK для доступу до нативних можливостей пристрою (камера, сповіщення, сенсори) без необхідності писати нативний

код на Swift чи Kotlin. На відміну від bare React Native, Expo керує конфігурацією нативних проєктів автоматично через систему плагінів, що спрощує обслуговування і оновлення залежностей. Expo Router реалізує файлову маршрутизацію за аналогією з Next.js: структура директорій app/ безпосередньо визначає навігаційне дерево застосунку, усуваючи потребу в ручному описі маршрутів.

2.2.2 Хмарна платформа та серверна частина

Supabase — відкрита альтернатива Firebase на базі PostgreSQL. Платформа надає такі сервіси:

- реляційна база даних PostgreSQL з повноцінною підтримкою SQL;
- автентифікація (email/пароль, OAuth, magic link);
- REST та GraphQL API, автоматично згенеровані з схеми БД;
- Realtime — двостороння синхронізація змін у таблицях через WebSocket;
- Storage — зберігання файлів і зображень;

Supabase розгорнуто на хмарній інфраструктурі AWS і масштабується автоматично. Вибір Supabase замість Firebase обумовлений повноцінною підтримкою реляційних запитів, відкритим вихідним кодом та можливістю самостійного розгортання.

2.2.3 Характеристика системи управління базами даних

PostgreSQL — потужна об'єктно-реляційна СУБД з відкритим вихідним кодом. У контексті розроблюваної системи PostgreSQL відіграє центральну роль: зберігає всі медико-біологічні дані, забезпечує транзакційну цілісність і реалізує RLS.

2.2.4 Управління станом, локалізація та допоміжні бібліотеки

TanStack Query реалізує декларативне управління асинхронним станом: кешування, фонове оновлення, повторні спроби та синхронізацію запитів між компонентами. Бібліотека суттєво спрощує взаємодію клієнта з API та усуває потребу в ручному управлінні станами завантаження й помилок.

Zustand — мінімалістична бібліотека глобального стану для React та React Native. На відміну від Redux, вона не вимагає шаблонного коду і надає простий API для створення сховищ. У розроблюваному застосунку Zustand використовується для зберігання поточного профілю дитини та налаштувань сесії.

i18next — фреймворк інтернаціоналізації для JavaScript, що забезпечує підтримку декількох мов у застосунку. Застосунок підтримує українську та англійську локалізацію з автоматичним визначенням мови пристрою та можливістю ручного перемикавання.

react-native-gifted-charts — бібліотека графіків для React Native, що використовується для відображення антропометричних даних дитини та стандартних кривих ВООЗ. Бібліотека надає гнучкий API для побудови лінійних і комбінованих графіків безпосередньо на нативному Canvas.

2.3 Механізми автентифікації та захисту даних

Безпека медико-біологічних даних дитини є критичною вимогою системи. Розроблювана архітектура реалізує захист на двох незалежних рівнях: на рівні транспорту всі запити передаються виключно через HTTPS, а на рівні СУБД доступ до рядків таблиць обмежується механізмом RLS.

Автентифікація здійснюється через Supabase Auth, що видає підписаний JWT-токен після успішного входу. Цей токен передається в заголовок кожного API-запиту та використовується СУБД для визначення

поточного користувача.

2.3.1 Row-Level Security та ізоляція даних користувачів

RLS — механізм PostgreSQL, що дозволяє визначати умови видимості рядків таблиці на рівні СУБД. При увімкненому RLS кожен SQL-запит автоматично отримує додаткові умови фільтрації, визначені відповідними політиками безпеки.

У розроблюваній системі кожна таблиця (профілі дітей, активності, вимірювання, вакцинації) має набір RLS-політик, що дозволяють користувачу читати та змінювати лише власні записи. Ідентифікатор користувача витягується безпосередньо з JWT-токена функцією `auth.uid()`, що унеможлиблює підробку ідентичності навіть за наявності прямого доступу до API.

Таким чином, RLS надає додатковий рівень захисту, незалежний від логіки застосунку: навіть якщо клієнтський код містить помилку авторизації, СУБД не поверне чужих даних.

2.3.2 Безпека мобільного застосунку

Мобільний застосунок, що обробляє персональні медичні дані, є потенційною ціллю для реверс-інжинірингу та несанкціонованого доступу. Для захисту на рівні клієнтського середовища виконання інтегровано два інструменти.

freeRASP — бібліотека захисту мобільного застосунку в режимі виконання (*Runtime Application Self-Protection*). Вона виявляє загрозливе середовище: root-доступ або jailbreak на пристрої, підключені дебагери, запуск у емуляторі, а також ознаки переупакування або підробки застосунку. У разі виявлення загрози застосунок може заблокувати виконання критичних операцій або завершити сесію.

Sentry — платформа моніторингу помилок і продуктивності. Інтеграція Sentry дозволяє в реальному часі отримувати звіти про необроблені винятки, падіння застосунку та повільні мережеві запити. Це суттєво скорочує час виявлення та виправлення дефектів у виробничому середовищі.

2.4 Стандарти ВООЗ та педіатричний моніторинг

ВООЗ у 2006 році опублікувала Стандарти зростання дитини (*Child Growth Standards*) — набір перцентильних кривих, що описують нормальний фізичний розвиток дітей від народження до п'яти років. Стандарти побудовані на основі міжнародного мультицентрового дослідження за участю дітей із шести країн і вважаються глобальним еталоном.

Для кожного показника (маса тіла, зріст, окружність голови) та кожної статі визначено п'ять перцентильних ліній: P3, P15, P50, P85 та P97. Значення P50 відповідає медіані популяції, P3 — нижній межі норми, P97 — верхній. Відхилення вимірювання за межі P3 або P97 є підставою для консультації з педіатром.

У розроблюваному застосунку перцентильні таблиці ВООЗ вбудовано статично для діапазону 0–24 місяці окремо для хлопчиків і дівчаток. При додаванні нового вимірювання застосунок автоматично визначає перцентиль дитини і відображає її положення на графіку відносно кривих норми. Статичне вбудовування таблиць усуває залежність від мережевого підключення при перегляді антропометричних даних.

2.5 Ключові алгоритми обробки медико-біологічних даних

2.5.1 Алгоритм ШІ-асистента на основі RAG

ШІ-асистент реалізує педіатричні консультації на основі технології Retrieval-Augmented Generation (RAG) — підходу, що поєднує векторний пошук у базі знань із генеративною мовною моделлю. На відміну від чистої генерації, RAG обмежує відповіді контекстом верифікованих педіатричних документів, що суттєво знижує ризик галюцинацій та невірних медичних рекомендацій.

Алгоритм складається з п'яти послідовних кроків, схему яких наведено на рис. 2.2.

На першому кроці запит користувача перетворюється на числовий вектор (embedding) за допомогою моделі `nomic-embed-text`. Цей вектор кодує семантичний зміст запиту у просторі фіксованої розмірності та є основою для подальшого пошуку.

На другому кроці вектор запиту використовується для пошуку найближчих сусідів у сховищі векторних embedding педіатричної бази знань, що зберігаються у розширенні `pgvector` бази даних PostgreSQL. Пошук виконується за косинусною подібністю та повертає п'ять найбільш релевантних фрагментів.

Отримані фрагменти формують RAG-контекст, що поєднується із системним промптом. Системний промпт визначає роль асистента, мову відповіді та обмеження компетенції.

Сформований промпт передається до Groq API — сервісу швидкого виведення великих мовних моделей (LLM) на апаратному прискорювачі LPU. Groq забезпечує швидкість генерації до 500 токенів/с, що дозволяє реалізувати стрімінгові відповіді без помітних затримок. Відповідь

LLM повертається клієнту через потоковий Supabase Edge Function і відображається у чаті в режимі реального часу.

Детальна реалізація серверної функції та логіки запитів розглянута у підрозділі 3.2.



Рисунок 2.2 – Схема алгоритму роботи ШІ-асистента на основі RAG

2.5.2 Алгоритм перцентильної класифікації антропометричних показників

Для оцінювання фізичного розвитку дитини застосунок реалізує алгоритм класифікації антропометричних показників на основі перцентильних таблиць ВООЗ. Алгоритм приймає на вхід значення показника (маса тіла, зріст або окружність голови), вік дитини в місяцях та стать, а повертає перцентильну зону та інтерпольовані значення ліній P3, P15, P50, P85, P97.

Ключовим кроком є лінійна інтерполяція між двома суміжними рядками таблиці для точного обчислення перцентилів при довільному віці. Результат класифікації визначає одну з п'яти зон: нижче P3, P3–P15, P15–P85 (норма), P85–P97 або вище P97. Відхилення за межі P3 або P97 є підставою для консультації з педіатром.

Перцентильні таблиці ВООЗ вбудовано статично у застосунок для діапазону 0–24 місяці окремо для хлопчиків і дівчаток, що усуває залежність від мережевого підключення. Принципову схему алгоритму наведено у додатку А.

ВИСНОВОК ДО РОЗДІЛУ 2

У другому розділі визначено технологічний стек та архітектурні принципи розроблюваної системи. Обґрунтовано вибір клієнт-серверної архітектури з чотирма логічними шарами: представлення, бізнес-логіки, даних і хмарного бекенду. Показано, що кросплатформна нативна розробка є оптимальним підходом для охоплення iOS та Android при єдиній кодовій базі.

Обрано React Native та Expo як основу клієнтської частини. React Native забезпечує нативне відображення компонентів без WebView, а Expo спрощує доступ до платформних можливостей і управління конфігурацією нативних проєктів через систему плагінів.

Хмарну платформу Supabase на базі PostgreSQL обрано завдяки підтримці реляційних запитів, механізму RLS для ізоляції даних між користувачами та вбудованому сервісу синхронізації в реальному часі.

Для управління станом обрано TanStack Query — для асинхронних запитів з автоматичним кешуванням, та Zustand — для глобального стану застосунку. Локалізацію забезпечує i18next з підтримкою української та англійської мов, а react-native-gifted-charts використовується для побудови антропометричних графіків із кривими BOOЗ. Захист виконуваного середовища реалізовано через freeRASP, моніторинг помилок — через Sentry.

Розглянуто стандарти BOOЗ щодо фізичного розвитку дітей і способів їх статичної інтеграції у застосунок у вигляді перцентильних таблиць для діапазону 0–24 місяці.

Описано ключові алгоритми системи: алгоритм ІІІ-асистента на основі RAG з використанням nomic-embed-text, pgvector та Groq API, а також алгоритм перцентильної класифікації антропометричних показників

					ІАЛЦ.467200.003 ПЗ	Аркуш
						26
Зм.	Лист	№ докум.	Підп.	Дата		

за стандартами ВООЗ. Реалізацію цих алгоритмів розглянуто у третьому розділі.

РОЗДІЛ 3

ПРОЕКТУВАННЯ ТА РЕАЛІЗАЦІЯ

КРОСПЛАТФОРМНОЇ СИСТЕМИ АГРЕГАЦІЇ

МЕДИКО-БІОЛОГІЧНИХ ДАНИХ

3.1 Архітектура системи

3.1.1 Загальна архітектурна схема застосунку

Розроблювана система є кросплатформним мобільним застосунком із хмарним бекендом. Система складається з двох основних частин: клієнтського мобільного застосунку, що функціонує на платформах iOS та Android, та хмарного бекенду на базі Supabase. Такий розподіл відповідає архітектурному патерну «тонкий клієнт», де уся логіка збереження та читання даних, перевірка прав доступу і бізнес-правила зосереджені на стороні сервера, а клієнт лише відображає дані та надсилає запити.

Клієнтський застосунок реалізовано засобами React Native та Expo. Він не містить власної серверної логіки та взаємодіє з хмарним бекендом виключно через REST API та WebSocket-підписки. Такий підхід дозволяє розгортати оновлення бекенду незалежно від оновлень застосунку в магазинах додатків, що суттєво скорочує час між виправленням серверної помилки та доступністю виправлення для кінцевих користувачів. Крім того, відсутність серверної логіки на клієнті зменшує поверхню атаки: навіть за умови компрометації клієнтського коду зловмисник не отримає доступу до даних іншого користувача.

Хмарний бекенд побудовано на платформі Supabase і містить чотири функціональних компоненти: PostgreSQL як основне сховище даних,

Supabase Auth для управління автентифікацією та сесіями, Supabase Realtime для синхронізації між пристроями в реальному часі та Supabase Storage для зберігання медіафайлів. Кожен із компонентів розгортається і обслуговується платформою Supabase, що звільняє розробника від необхідності адміністрування серверної інфраструктури.

Взаємодія між клієнтом і бекендом здійснюється через `supabase-js` — офіційний JavaScript-клієнт Supabase, що інкапсулює формування HTTP-запитів, управління JWT-токенами та підтримку Realtime-підписок. Усі запити передаються по захищеному каналу HTTPS; для WebSocket-з'єднань використовується протокол WSS. Завдяки типізованому клієнту з автоматично згенерованими TypeScript-інтерфейсами схеми бази даних будь-яка невідповідність між клієнтськими запитами та фактичною схемою виявляється на етапі компіляції, а не в середовищі продакшну.

При першому запуску застосунок перевіряє наявність збереженої сесії в `AsyncStorage`. Якщо сесія дійсна, застосунок переходить до головного екрану; в іншому випадку відображається екран автентифікації. Після успішного входу Supabase Auth видає пару tokenів: JWT доступу (термін дії 1 година) та token оновлення. Автоматична ротація tokenів відбувається при переведенні застосунку на передній план, що унеможливорює тривале використання скомпрометованого tokenа доступу.

3.1.2 Архітектурні особливості клієнтської частини

Клієнтський застосунок організовано за принципом `feature-based` архітектури: увесь код згруповано по функціональних доменах, а не за технічними ролями. Така структура забезпечує зв'язність коду усередині кожного модуля і мінімізує залежності між модулями, що значно спрощує навігацію по кодовій базі та локалізацію помилок. На відміну від

традиційного підходу, де компоненти, хуки та утиліти одного домену розкидані по різних директоріях, feature-based підхід концентрує весь код певної функціональності в одному місці.

Директорія `app/` містить навігаційні екрани у форматі `Expo Router`, де структура директорій безпосередньо відображається у дерево маршрутів застосунку. Директорія `features/` зберігає доменну логіку восьми функціональних областей: `feedings`, `sleeps`, `diapers`, `measurements`, `milestones`, `vaccinations`, `children` та `auth`. Директорія `components/` містить перевикористовувані компоненти UI, що не належать до жодного конкретного домену. Директорії `stores/`, `lib/`, `constants/` та `i18n/` зберігають глобальні стани, клієнтські інтеграції, дизайн-токени та файли локалізації відповідно. Така організація дозволяє розробнику, що не знайомий з проектом, за назвою директорії відразу зрозуміти, яку функціональність вона реалізує, що знижує поріг входу для нових учасників команди.

Кожна директорія в `features/` містить файл `queries.ts` з React-хуками для отримання і мутації даних через `TanStack Query` та `Supabase`. Хуки отримання даних побудовані на `useQuery` і повертають стандартизовані об'єкти з полями `data`, `isLoading`, `isError` та `refetch`, що забезпечує однорідний інтерфейс взаємодії з даними по всьому застосунку. Хуки мутацій побудовані на `useMutation` і автоматично інвалідують відповідні ключі кешу після успішного виконання операції, завдяки чому UI оновлюється без явного виклику `refetch` з боку компонентів.

Шар представлення відокремлений від шару даних: екрани у `app/` отримують дані виключно через хуки з `features/`, а не звертаються до `Supabase` напряму. Це дозволяє тестувати бізнес-логіку незалежно від компонентів UI і спрощує заміну джерела даних без переписування екранів. Наприклад, якщо у майбутньому виникне необхідність замінити `Supabase`

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		30

на інший бекенд, зміни знадобляться лише у файлах `queries.ts` кожного модуля, тоді як усі екрани і компоненти залишаться незмінними.

3.1.3 Архітектура хмарної серверної частини

Хмарний бекенд системи побудовано на базі платформи Supabase, що надає повний набір інфраструктурних сервісів без необхідності розробки та підтримки власного серверного застосунку. Такий підхід дозволяє зосередити весь обсяг розробки на клієнтській логіці та предметній області застосунку, не витрачаючи ресурси на розробку і підтримку типових серверних компонентів, таких як автентифікація, авторизація, файлове сховище та синхронізація в реальному часі. Усі компоненти бекенду автоматично масштабуються залежно від навантаження без участі розробника.

Центральним компонентом бекенду є PostgreSQL — реляційна СУБД, яка зберігає всі предметні дані системи та реалізує ізоляцію даних між користувачами через механізм RLS (Row-Level Security), детально описаний у підрозділі 3.2.2. Поверх PostgreSQL розгорнуто PostgREST — проміжний шар, що автоматично генерує REST API для кожної таблиці і представлення бази даних. Клієнтський код звертається до бази даних через `supabase-js`, який транслює типізовані запити у відповідні HTTP-запити до PostgREST, а той виконує відповідний SQL-запит від імені аутентифікованого користувача.

Supabase Auth реалізує управління обліковими записами і сесіями. Сервіс підтримує реєстрацію і вхід через email та пароль, а також авторизацію через Google за протоколом OAuth 2.0. OAuth (Open Authorization) — відкритий стандарт делегованої авторизації, що дозволяє користувачу надати застосунку обмежений доступ до свого акаунту у стороннього провайдера (Google, Apple тощо) без передачі пароля: замість цього провайдер видає одноразовий код, який застосунок обмінює на

					ІАЛЦ.467200.003 ПЗ	Аркуш
						31
Зм.	Лист	№ докум.	Підп.	Дата		

токени доступу. При успішній автентифікації Supabase Auth генерує підписані JWT-токени, що містять ідентифікатор користувача у полі `sub` та метадані сесії. Ці токени передаються у заголовок `Authorization: Bearer` кожного запиту до PostgREST, де PostgreSQL витягує ідентифікатор користувача функцією `auth.uid()` для застосування RLS-фільтрів. Завдяки такій інтеграції авторизаційна перевірка відбувається безпосередньо при виконанні SQL-запиту, а не у додатковому прошарку між клієнтом і базою даних.

Supabase Realtime транслює зміни у таблицях бази даних клієнтам через постійне WebSocket-з'єднання на основі механізму логічної реплікації PostgreSQL, забезпечуючи миттєву синхронізацію між пристроями без регулярного опитування сервера. Детальна реалізація підписок описана у підрозділі 3.3.4.

3.1.4 Архітектура бази даних

База даних системи реалізована у PostgreSQL і містить десять таблиць. Детальну ER-діаграму бази даних наведено у додатку Д2. Всі таблиці використовують ідентифікатори типу `uuid`, що генеруються функцією `gen_random_uuid()`, — на відміну від автоінкрементних цілочисельних ідентифікаторів, `UUID` не дозволяють атакуючому передбачити ідентифікатор запису іншого користувача та виконати цільований запит на його отримання.

Центральною сутністю є таблиця `children`, що зберігає профілі дітей: ідентифікатор, повне ім'я, дата народження, стать, нотатки та посилання на аватар у Supabase Storage. Зв'язок між профілями дітей та користувачами системи реалізовано через окрему таблицю `caregivers`, що містить пару (`profile_id`, `child_id`) та роль доглядача (`owner`, `co_parent`, `caregiver` або `pediatrician`). Такий підхід реалізує відношення «багато-до-багатьох»

між користувачами і дітьми та дозволяє одній дитині мати кількох доглядачів з різними рівнями доступу, що відповідає реальному сімейному сценарію використання застосунку.

Таблиці `feedings` та `sleeps` зберігають події з тимчасовими мітками початку та завершення; таблиця `diapers` фіксує одиничні події з єдиною міткою часу (`occurred_at`). Таблиця `feedings` містить поле `kind` з перерахуванням допустимих типів годування (`breast_left`, `breast_right`, `bottle_breast_milk`, `bottle_formula`, `solid`), а також поля тривалості сеансу та кількості молока в мілілітрах. Для тривалих активностей — годування та сон — реалізовано підтримку активних сеансів: запис зі значенням `NULL` у полі `ended_at` вважається незавершеним поточним сеансом, що дозволяє відображати активний таймер на головному екрані без зберігання проміжного стану у клієнтському застосунку.

Таблиця `growth_measurements` зберігає антропометричні вимірювання дитини з типом показника (`weight`, `height`, `head_circumference`), числовим значенням та одиницею вимірювання. Таблиця `milestones` фіксує факти досягнення вікових міток з посиланням на шаблони (`milestone_templates`), що зберігаються в окремій системній таблиці, заповненій стандартними педіатричними даними. Таблиця `vaccinations` зберігає факти вакцинацій без шаблонів: назву та код вакцини у текстових полях `vaccine_name` та `vaccine_code` безпосередньо у записі. Розподіл на таблиці шаблонів та таблиці досягнень дозволяє централізовано оновлювати набір контрольних точок розвитку без необхідності міграції індивідуальних записів кожного користувача.

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		33

3.2 Реалізація серверної частини

3.2.1 Проєктування та реалізація бази даних

Схема бази даних розроблена з урахуванням вимог нормалізації до третьої нормальної форми та особливостей механізму RLS у Supabase. Дотримання нормалізації усуває надлишковість даних: наприклад, інформація про шаблони вікових досягнень зберігається один раз у таблиці `milestone_templates`, а таблиця `milestones` містить лише зовнішній ключ на відповідний шаблон. Усі предметні таблиці містять первинний ключ типу `uuid` з автоматичною генерацією через `gen_random_uuid()`, що унеможливлює передбачення ідентифікаторів атакуючим та спрощує горизонтальне масштабування бази даних у майбутньому.

Таблиця `children` є головною сутністю схеми і містить такі поля: `id uuid PRIMARY KEY DEFAULT gen_random_uuid()`, `full_name text NOT NULL`, `date_of_birth date NOT NULL`, `sex text CHECK (sex IN ('male', 'female', 'unspecified'))`, `notes text`, `avatar_url text` та `created_at timestamptz DEFAULT now()`. Обмеження `CHECK` на поле `sex` забезпечує цілісність даних на рівні СУБД і не дозволяє клієнту зберегти довільне рядкове значення для цього поля. Таблиця `caregivers` реалізує зв'язок між користувачами та дітьми через поля `profile_id uuid REFERENCES auth.users NOT NULL` та `child_id uuid REFERENCES children NOT NULL` з унікальним обмеженням на цю пару, що запобігає дублюванню записів про одного й того самого доглядача.

Таблиця `feedings` містить такі ключові поля: `child_id uuid REFERENCES children NOT NULL`, `kind text NOT NULL`, `started_at timestamptz NOT NULL`, `ended_at timestamptz` (зі значенням `NULL` для активного сеансу), `amount_ml numeric(6,1)` для кількості молока у

пляшковому годуванні та `created_by` `uuid REFERENCES auth.users` для фіксації автора запису. Таблицю `sleeps` побудовано за аналогічним принципом; таблиця `diapers` використовує єдину мітку часу `occurred_at` замість пари `started_at/ended_at`. У таблиці `growth_measurements` додано поле `value numeric(7,2) NOT NULL` для числового значення та `unit text` для одиниці вимірювання, що забезпечує точність до двох десяткових знаків та підтримує зберігання як метричних, так і імперіальних одиниць.

При створенні нової дитини база даних автоматично додає запис у таблицю `caregivers` завдяки тригеру `on_child_created`, що виконується після кожної вставки у таблицю `children`. Тригер визначає поточного аутентифікованого користувача через `auth.uid()` та додає пару `(auth.uid(), new.id)` з роллю `owner`. Такий підхід гарантує, що кожна дитина завжди має хоча б одного власника, і спрощує клієнтський код: для реєстрації нової дитини потрібен лише один `INSERT`-запит до таблиці `children`, а зв'язок із поточним користувачем створюється автоматично на стороні СУБД без додаткових запитів з клієнта.

3.2.2 Реалізація Row-Level Security

RLS є ключовим механізмом ізоляції даних у системі. Після ввімкнення RLS для таблиці PostgreSQL автоматично застосовує визначені політики до кожного запиту незалежно від того, яким способом цей запит надійшов — через API, прямий SQL-клієнт або будь-який інший інтерфейс. Це означає, що захист даних реалізований не на рівні логіки застосунку, а на рівні СУБД, і не може бути обійдений навіть при помилках у клієнтському коді. Для таблиці `children` визначено дві політики. Перша дозволяє читання лише тим користувачам, для яких існує відповідний запис у таблиці `caregivers`:

```
CREATE POLICY "caregivers can view children"
ON children FOR SELECT
USING (
    EXISTS (
        SELECT 1 FROM caregivers
        WHERE caregivers.child_id = children.id
        AND caregivers.profile_id = auth.uid()
    )
);
```

Друга політика дозволяє вставку будь-якому аутентифікованому користувачеві, оскільки тригер `on_child_created` автоматично призначить його власником щойно створеного профілю дитини:

```
CREATE POLICY "authenticated users can create children"
ON children FOR INSERT
WITH CHECK (auth.uid() IS NOT NULL);
```

Для таблиць активностей (`feedings`, `sleeps`, `diapers`, `growth_measurements`) RLS-політики перевіряють доступ через таблицю `caregivers` аналогічно до таблиці `children`, але умова перевіряє `child_id` запису активності. Наприклад, для `feedings` SELECT-політика виглядає так:

```
CREATE POLICY "caregivers can view feedings"
ON feedings FOR SELECT
USING (
    EXISTS (
        SELECT 1 FROM caregivers
        WHERE caregivers.child_id = feedings.child_id
        AND caregivers.profile_id = auth.uid()
    )
);
```


)
);

Аналогічні політики для операцій INSERT, UPDATE та DELETE гарантують, що користувач може змінювати лише дані тих дітей, доглядачем яких він є. Функція `auth.uid()` повертає ідентифікатор поточного користувача з JWT-токена, переданого в заголовок запиту; якщо токен відсутній або недійсний, функція повертає NULL, і жодна RLS-політика не виконується, тобто запит від неаутентифікованого користувача автоматично повертає порожній результат для всіх захищених таблиць.

3.3 Реалізація клієнтської частини

3.3.1 Організація структури модулів

Реалізація клієнтської частини дотримується єдиного шаблону в усіх функціональних модулях: кожна директорія у `features/` реалізує повний вертикальний зріз одного домену — від типів та запитів до компонентів відображення. Шар даних кожного модуля будується на основі TanStack Query і складається з файлу `queries.ts` із хуками отримання даних (`useQuery`) та мутацій (`useMutation`), а також окремих файлів для типів і утиліт домену. Ієрархічна система ключів кешу забезпечує автоматичну інвалідацію при мутаціях без ручного виклику перезавантаження даних з боку компонентів.

Наприклад, модуль годувань містить у файлі `features/feedings/queries.ts` шість хуків: `useFeedingsForDay` та `useFeedingsInRange` для отримання записів за різними часовими діапазонами, `useActiveFeeding` для відстеження незавершеного сеансу, а також `useStartFeeding`, `useStopFeeding` та `useCreateFeeding` для операцій запису. Ключі кешу побудовано ієрархічно: `['feedings',`

					ІАЛЦ.467200.003 ПЗ	Аркуш
						37
Зм.	Лист	№ докум.	Підп.	Дата		

`childId]` для всіх записів дитини та `['feedings', childId, 'day', date]` для записів конкретного дня. Завдяки цій ієрархії мутація `useCreateFeeding` може одночасно інвалідувати кеш поточного дня і загальний кеш дитини одним викликом з шаблоном `['feedings', childId]`, що охоплює всі підпорядковані ключі.

Спільні компоненти UI зосереджені в директорії `components/` і поділені на логічні групи залежно від призначення. Контейнерні компоненти `ScreenContainer` та `FormScreen` реалізують стандартні макети екранів з урахуванням безпечних зон пристрою (`SafeAreaView`) та підтримки прокручування. Компоненти форм `AppTextInput`, `DateField` та `KindGrid` забезпечують єдиний візуальний стиль полів введення даних по всьому застосунку. Графічні компоненти `DailyBarChartCard` та `MeasurementChartCard` інкапсують логіку побудови та відображення графіків. Усі компоненти параметризовані через TypeScript-інтерфейси та підтримують темну і світлу теми через хук `useTheme` бібліотеки `React Native Paper`. Для реалізації плавних анімацій у спільних компонентах використовується бібліотека `react-native-reanimated`, що виконує анімації безпосередньо у нативному потоці без передачі даних через JavaScript bridge на кожному кадрі. Завдяки цьому анімації зберігають частоту 60 кадрів на секунду навіть під час інтенсивного оновлення UI. `Reanimated` застосовується у `BottomSheet` — модальному аркуші, що виїжджає знизу екрана, де `withSpring` та `withTiming` анімують переміщення панелі по вертикалі та прозорість фону, — та у `ActiveChildPanel`, де `withSpring` анімує індикатор активної дитини при переключенні між профілями. Спільні значення (`useSharedValue`) зберігаються у нативному потоці, а стилі обчислюються через `useAnimatedStyle` без участі JS-рантайму під час відтворення анімації.

Локалізація реалізована бібліотекою `i18next` у поєднанні з

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		38

react-i18next із файлами перекладів у директорії i18n/. Застосунок підтримує дві мови: українську (за замовчуванням) та англійську. Обрана мова зберігається у AsyncStorage та застосовується при наступному запуску без необхідності повторного вибору. Всі рядки у компонентах UI отримуються через хук useTranslation, що забезпечує централізоване управління текстовим контентом і спрощує додавання нових мов у майбутньому.

Статична типізація всього проєкту забезпечується TypeScript у суворому режимі ("strict": true у tsconfig.json), що вмикає перевірку на null/undefined, заборону неявних типів any та суворий контроль типів параметрів функцій. Псевдонім шляху @/* дозволяє імпортувати будь-який модуль відносно кореня проєкту без відносних шляхів виду ../../../../, що скорочує кількість помилок при переміщенні файлів. Статичний аналіз коду виконується ESLint із пресетом eslint-config-expo, який охоплює правила для React, React Native, TypeScript та доступності UI-компонентів. Разом TypeScript і ESLint утворюють двошаровий бар'єр якості: компілятор відловлює структурні помилки типів, а лінтер — проблеми стилю, небезпечні патерни та порушення правил хуків React.

3.3.2 Моніторинг помилок та безпека середовища виконання

Для відстеження збоїв і аномалій у продакшн-середовищі інтегровано бібліотеку @sentry/react-native. Sentry — це платформа моніторингу помилок, що автоматично перехоплює необроблені винятки, збої JavaScript-рантайму та відмови нативних модулів, після чого передає структуровані звіти на хмарний дашборд із повним стек-трейсом, інформацією про пристрій, версію застосунку та контекст користувача. Ініціалізація виконується одноразово при запуску застосунку:

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		39

```
Sentry.init({
  dsn: process.env.EXPO_PUBLIC_SENTRY_DSN,
  tracesSampleRate: 0.2,
});
```

Параметр `tracesSampleRate: 0.2` означає, що трасування продуктивності збирається для 20% сесій, що дозволяє виявляти повільні мережеві запити та вузькі місця рендерингу без суттєвого навантаження на інфраструктуру. DSN-адреса (ідентифікатор проєкту Sentry) передається через змінну оточення `EXPO_PUBLIC_SENTRY_DSN` і не зберігається в репозиторії.

Для захисту від запуску застосунку у скомпрометованому середовищі інтегровано бібліотеку `freerasp-react-native`. Вона виконує перевірки цілісності середовища виконання: виявляє jailbreak (iOS) та root-доступ (Android), присутність hooking-фреймворків (Frida, Xposed), запуск у емуляторі, а також модифікацію бінарного файлу застосунку. При виявленні загрози бібліотека передає подію до Sentry для фіксації у системі моніторингу та виконує налаштовану реакцію. Така інтеграція утворює дворівневий захист: Sentry фіксує помилки у штатному режимі, а freeRASP реагує на цілеспрямовані спроби маніпуляцій із застосунком.

3.3.3 Навігаційна структура та управління станом

Навігаційна структура застосунку реалізована засобами Expo Router, що відображає файлову систему директорії `app/` безпосередньо у дерево маршрутів. Такий підхід усуває необхідність ручної реєстрації маршрутів: додавання нового файлу `app/feedings/new.tsx` автоматично робить маршрут `/feedings/new` доступним у навігаційному стеку. Детальну структурну схему системи наведено у додатку ДЗ. Expo

Router забезпечує також типобезпечну навігацію: при виклику `router.push('/feedings/new')` TypeScript перевіряє наявність відповідного файлу у директорії `app/`, що виключає помилки навігації на етапі компіляції.

Кореневий файл `app/_layout.tsx` є точкою входу навігаційного стеку. Він обгортає все дерево компонентів у провайдери: `AuthProvider` для управління сесією, `QueryClientProvider` для `TanStack Query` та `PaperProvider` для теми матеріального дизайну. На цьому ж рівні ініціалізуються `Realtime`-підписки та компонент `AuthGate`, що реалізує логіку охорони маршрутів. Основний стек містить два вкладені навігатори: вкладкову навігацію з трьома головними екранами («Головна», «Статистика», «Налаштування») та модальний стек із формами введення даних, що відображаються поверх вкладок як стандартні модальні вікна платформи.

Глобальний стан застосунку зберігається у двох `Zustand`-сховищах з персистентністю через `AsyncStorage`. Сховище `activeChild` містить ідентифікатор поточно обраної дитини; при зміні цього ідентифікатора всі `React Query` хуки, що залежать від `childId`, автоматично виконують повторний запит до API. Такий реактивний ланцюжок забезпечує миттєве оновлення всього інтерфейсу при переключенні між профілями дітей без явного перезавантаження будь-якого екрана. Сховище `themePreference` зберігає обраний режим теми (`system`, `light` або `dark`) і передається в `PaperProvider` для застосування відповідної теми матеріального дизайну.

3.3.4 Зберігання файлів та синхронізація в реальному часі

Supabase Storage використовується для зберігання аватарів дітей. При завантаженні зображення застосунок обирає фото з галереї пристрою через `expo-image-picker` та стискає його до максимального розміру 512×512 пікселів для зменшення обсягу переданих даних. Стиснуте зображення завантажується у бакет `avatars` за детермінованим шляхом `{child_id}/avatar.jpg`, після чого публічний URL зображення зберігається у полі `avatar_url` таблиці `children`. Використання детермінованого шляху замість генерованого імені файлу дозволяє автоматично перезаписувати попередній аватар при завантаженні нового без необхідності видалення старого файлу та оновлення посилання.

Синхронізація між пристроями реалізована через Supabase Realtime у файлі `lib/realtime.ts`. При активації основного стеку застосунок підписується на канал `child:{activeChildId}`, що відслідковує зміни у таблицях активностей конкретної дитини. Фільтр на рівні каналу обмежує трафік: клієнт отримує лише події, де `child_id` збігається з ідентифікатором активної дитини, а не всі зміни у таблиці. При отриманні події `postgres_changes` React Query клієнт інвалідує відповідний ключ кешу, що ініціює фонове оновлення даних:

supabase

```
.channel(`child:${childId}`)
.on('postgres_changes',
  { event: '*', schema: 'public', table: 'feedings',
    filter: `child_id=eq.${childId}` },
  () => queryClient.invalidateQueries(
    { queryKey: ['feedings', childId] })
)
```

.subscribe();

При зміні активної дитини у сховищі `activeChild` попередня підписка скасовується, а нова підписка встановлюється для нового ідентифікатора. Такий підхід запобігає накопиченню неактуальних підписок та зайвому мережевому трафіку. Завдяки реалізованому механізму синхронізації, якщо один із батьків додає запис про годування зі свого пристрою, застосунок іншого батька оновлює відображення автоматично протягом 1–2 секунд без необхідності ручного оновлення сторінки.

3.3.5 Реалізація клієнтської автентифікації

Автентифікація реалізована через `AuthProvider` — React-контекст, що обгортає весь застосунок і надає сесію та методи автентифікації всім дочірнім компонентам. При монтуванні провайдер викликає `supabase.auth.getSession()` для отримання збереженої сесії з `AsyncStorage` пристрою. Якщо сесія знайдена, провайдер перевіряє час її закінчення: якщо до закінчення залишається менше 60 хвилин, виконується автоматичне оновлення токена через `supabase.auth.refreshSession()`. У разі невдачі оновлення (наприклад, коли токен відкликано з боку сервера) сесія вважається недійсною і користувача перенаправляють на екран входу.

Для реєстрації та входу через email і пароль реалізовано відповідні мутації у файлі `features/auth/mutations.ts`. Помилки автентифікації, що повертаються Supabase Auth, перехоплюються і відображаються у формі у вигляді зрозумілих повідомлень для користувача: «Невірний email або пароль», «Email вже зареєстровано» тощо. Вхід через Google реалізовано за допомогою OAuth 2.0 потоку з бібліотекою `expo-web-browser`: застосунок відкриває системний браузер із сторінкою авторизації Google, після успішного проходження якої браузер перенаправляє на `deep-link` застосунку

з одноразовим кодом авторизації, який `supabase-js` автоматично обмінює на пару токенів доступу.

Компонент `AuthGate` у кореневому `_layout.tsx` підписується на зміни стану автентифікації через `supabase.auth.onAuthStateChange` та умовно відображає або основний стек із вкладками, або стек автентифікації. Завдяки реактивній підписці стан навігації оновлюється миттєво при зміні сесії — як при вході, так і при виході. Екран автентифікації побудований за принципом єдиної точки входу: один екран обслуговує як вхід, так і перехід до реєстрації. Верхня частина екрана є брендовою зоною з градієнтним фіолетовим фоном та логотипом застосунку, нижня — білою картою з двома варіантами автентифікації: формою email/пароль та кнопкою «Продовжити з Google». Перемикач мови у правому верхньому куті дозволяє змінювати локалізацію безпосередньо на екрані входу без необхідності авторизації. Після успішної реєстрації застосунок автоматично виконує вхід і перенаправляє на екран створення першого профілю дитини.

```
supabase.auth.getSession().then(({ data }) => {
  setSession(data.session);
  setLoading(false);
});
const { data: sub } = supabase.auth.onAuthStateChange(
  (_event, s) => setSession(s)
);
return () => sub.subscription.unsubscribe();
```


3.3.6 Реалізація ключових екранів та компонентів

Головний екран застосунку (`app/(tabs)/index.tsx`) є центральним інформаційним хабом і першим, що бачить користувач після входу. У верхній частині відображається компонент `HeroCard` — вітальна картка з іменем поточного користувача та датою; нижче — картка активної дитини з аватаром, ім'ям та кількістю повних місяців від дати народження. Рядок лічильників `StatsRow` відображає три агреговані показники поточного дня: кількість годувань, загальний час сну та кількість змін підгузків. Якщо є активний незавершений сеанс, відображається також компонент `ActiveTimerCard` з таймером у реальному часі та кнопкою зупинки. Сітка швидких дій надає прямий доступ до форм введення всіх типів активностей, а хронологічна стрічка подій у нижній частині агрегує записи з усіх доменів у зворотному хронологічному порядку. Навігація по дням реалізована через кнопки «назад» та «вперед», що змінюють значення `selectedDay` через `subDays/addDays` з `date-fns`, дозволяючи переглядати стрічку будь-якого минулого дня без переходу на окремий екран.

Екран статистики (`app/(tabs)/stats.tsx`) відображає аналітику за останні 7 днів у вигляді трьох стовпчастих графіків: кількість годувань, загальний час сну та кількість змін підгузків по днях тижня. Компонент `DailyBarChartCard` для кожного показника отримує масив значень за 7 днів і будує графік засобами бібліотеки `react-native-gifted-charts`, де кожен стовпець відповідає одному дню. Для аналізу режиму сну реалізовано окремий компонент `SleepPatternChart`, де вісь Y відображає години доби від 00:00 до 24:00, а стовпці показують діапазон активного сну у кожен день, дозволяючи візуально виявляти порушення нічного режиму. Під графіком сну відображається підсумковий рядок із тривалістю денного та нічного сну за тиждень. Якщо середня кількість замінів підгузків виходить за межі норми

(менше 5 або більше 8 на добу), екран відображає відповідне попередження лікарського характеру.

Екран статистики підтримує експорт даних у формат PDF через функцію `exportStatsPdf` у файлі `lib/pdf.ts`. Функція динамічно завантажує бібліотеки `expo-print` та `expo-sharing` через `require()` замість статичного імпорту, що дозволяє екрану статистики коректно завантажуватись у Expo Go, де ці нативні модулі відсутні. При активації експорту функція `buildHtml` генерує повноцінний HTML-документ із CSS-стилями, який містить вітальну секцію з іменем дитини та датою, окремі картки для годувань, сну та підгузків із таблицями по днях, а також секцію антропометричних вимірювань. Бібліотека `expo-print` конвертує цей HTML у .pdf-файл засобами нативного рушія платформи (`Print.printToFileAsync`), а `expo-sharing` відкриває стандартний системний діалог передачі файлу (`Sharing.shareAsync`), через який користувач може зберегти PDF у Files, надіслати електронною поштою або передати через AirDrop. Всі текстові рядки та формати дат у PDF локалізовані відповідно до поточної мови застосунку через `i18next` та `date-fns`.

Форми введення даних побудовані на єдиному компоненті `FormScreen`, що реалізує стандартний макет із заголовком, кнопкою закриття та прокручуванням тілом з урахуванням клавіатури. Вибір типу активності реалізований через компонент `KindGrid` — сітку іконок-плиток, де кожен варіант відображається у вигляді картки з кольоровою іконкою та підписом, а активний елемент підсвічується кольоровою рамкою. Форма годування підтримує п'ять типів: ліву та праву груди, пляшку з грудним молоком, пляшку з сумішшю і тверду їжу, — а також два режими введення: запуск таймера в реальному часі кнопкою «Старт зараз» або ручне введення тривалості минулого сеансу. Форма підгузків підтримує чотири типи:

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		46

мокий (wet), брудний (dirty), змішаний (mixed) та сухий (dry), — що дозволяє точніше відстежувати стан дитини та виявляти можливі проблеми зі здоров'ям. Вибір дати і часу реалізований через `DatePicker`, що відкриває нативний системний пікер дати платформи, забезпечуючи звичний для користувача досвід.

```
export function useStartFeeding() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async ({ childId, kind }) => {
      const { data, error } = await supabase
        .from('feedings')
        .insert({ child_id: childId, kind,
          started_at: new Date().toISOString(),
          ended_at: null })
        .select('*').single();
      if (error) throw error;
      return data;
    },
    onSuccess: (f) =>
      qc.invalidateQueries({ queryKey: feedingsKey(f.child_id) }),
  });
}
```

Екран налаштувань (`app/(tabs)/settings.tsx`) організований у три секції. Секція «Профіль» надає доступ до редагування імені та email, а також до зміни пароля через відповідні модальні форми. Секція «Налаштування» містить перемикач мови (українська/англійська) та вибір теми оформлення. Секція «Акаунт» включає кнопки виходу та видалення облікового запису,

кожна з яких відкриває `ConfirmDialog` для підтвердження деструктивної операції. Зміна мови відбувається реактивно через хук `useLocale` без перезавантаження застосунку.

Екран вікових досягнень (`app/milestones/index.tsx`) відображає шаблони контрольних точок розвитку дитини, згрупованих за віковими діапазонами. Шаблони завантажуються з таблиці `milestone_templates` і збагачуються даними про фактичні досягнення дитини з таблиці `milestones`. Якщо вік дитини потрапляє у діапазон (`age_min_months`–`age_max_months`) шаблону, і досягнення ще не зафіксоване, шаблон потрапляє до секції «Актуально зараз» на початку екрана, привертаючи увагу батьків до поточних цілей розвитку. Кожен шаблон відображається як картка з категорійною іконкою, описом та статусом; натискання відкриває екран деталей, де можна зафіксувати дату досягнення та додати нотатку.

Екран вакцинацій (`app/vaccinations/index.tsx`) побудований аналогічно до екрана досягнень, але групує щеплення за педіатричними слотами (при народженні, 2 місяці, 4 місяці, 6 місяців тощо). Записи завантажуються з таблиці `vaccinations`, де зберігаються код та назва вакцини, дата введення (`administered_at`) та дата планового введення (`scheduled_for`). Секція «Актуально зараз» виокремлює записи, термін яких настав відповідно до поточного віку дитини. Кожна картка відображає назву вакцини та статус введення; при натисканні відкривається форма фіксації дати введення та нотатки.

Редагування профілю дитини доступне через екран `app/children/[id]/edit.tsx`, що дозволяє змінити ім'я, дату народження, стать та нотатки, а також завантажити або видалити аватар через `expo-image-picker`. Видалення акаунту виконується з екрана налаштувань і вимагає підтвердження через `ConfirmDialog`, після чого каскадно видаляються всі пов'язані записи завдяки `ON DELETE CASCADE` у схемі бази

даних.

3.3.7 Робота з датами та часом

Уся робота з датами та часом у застосунку виконується бібліотекою `date-fns` — модульною утилітою, що надає функції для форматування, арифметики та порівняння дат без мутації об'єктів і без глобального стану. На відміну від `moment.js`, `date-fns` підтримує `tree-shaking`: до фінального пакету потрапляють лише ті функції, що реально імпортуються, що зменшує розмір збірки. Бібліотека використовується у трьох ключових контекстах. По-перше, у функції `formatAge` (`lib/age.ts`) для обчислення віку дитини: `differenceInDays`, `differenceInMonths` та `differenceInYears` визначають точний вік і обирають правильну одиницю відображення (дні, тижні, місяці або роки). По-друге, у функції `aggregateByDay` (`lib/aggregate.ts`) для формування 7-денних бакетів статистики: `subDays` генерує масив дат від сьогодні назад, `parseISO` перетворює рядки з бази даних у об'єкти `Date`, а `isSameDay` зіставляє записи активностей із відповідним бакетом. По-третє, у модулі PDF-експорту (`lib/pdf.ts`) для локалізованого форматування дат у звіті через `format` із передачею об'єкта `locale` відповідно до поточної мови інтерфейсу.

3.3.8 Реалізація алгоритмів обробки медичних даних

Алгоритм класифікації антропометричних показників відповідно до стандартів ВООЗ реалізовано у файлі `features/measurements/who/index.ts`. Вихідні перцентильні таблиці зберігаються статично у `who/standards.ts` у вигляді вкладених числових масивів формату `[age, P3, P15, P50, P85, P97]`, де `age` — вік у місяцях окремо для кожної статі та кожного типу показника. Такий підхід дозволяє уникнути мережевих запитів при кожному розрахунку, оскільки

таблиці компілюються безпосередньо у пакет застосунку та залишаються незмінними між сесіями. Стандарти ВООЗ охоплюють вікові діапазони від 0 до 24 місяців для дівчаток та хлопчиків окремо, що дозволяє враховувати статеві відмінності у нормах фізичного розвитку.

Функція `whoBands(sex, kind, ageMonths)` виконує лінійну інтерполяцію між двома суміжними рядками таблиці для точного вікового значення дитини. Спочатку обирається відповідна таблиця за статтю та типом показника, потім знаходяться два сусідні рядки, між якими знаходиться вік дитини (`floor(ageMonths)` та `ceil(ageMonths)`). Для кожного з п'яти перцентилів (P3, P15, P50, P85, P97) обчислюється інтерпольоване значення за формулою:

$$v = v_0 + (v_1 - v_0) \times (ageMonths - month_0),$$

де v_0 та v_1 — значення перцентилів для нижнього та верхнього рядків таблиці, $month_0$ — нижній цілий вік у місяцях. Функція повертає об'єкт `BandValues` з інтерпольованими значеннями п'яти перцентилів, що використовуються для побудови зон на графіку та класифікації вимірювання.

Функція `classifyValue(value, bands)` визначає, в яку зону потрапляє виміряне значення: нижче P3, між P3 і P15, між P15 і P85 (норма), між P85 і P97, або вище P97. Результати класифікації відображаються на графіку кольоровими зонами: зелена відповідає нормі між P15 та P85, жовті — граничним значенням P3–P15 і P85–P97, червоні — патологічним відхиленням нижче P3 та вище P97. Текстовий підпис під графіком дублює класифікацію словесно, що полегшує сприйняття для батьків без медичної освіти.

```
export function classifyValue(
  value: number, bands: Bands
```

```

): PercentileBand {
    if (value < bands.p3) return 'below_p3';
    if (value < bands.p15) return 'p3_p15';
    if (value <= bands.p85) return 'p15_p85';
    if (value <= bands.p97) return 'p85_p97';
    return 'above_p97';
}

```

Алгоритм агрегації даних сну реалізовано у хуку useSleepPattern. Хук отримує список сесій сну за 7 днів і для кожного дня обчислює два показники: тривалість денного та нічного сну. Нічним вважається сон, що частково або повністю знаходиться в проміжку 19:00–06:59 наступного дня. Тривалість частини кожного сну, що перетинається з нічним вікном, обчислюється як довжина перетину двох часових відрізків, після чого значення підсумовуються для кожного дня окремо. Отримані дані передаються до компонента SleepPatternChart, який будує двовимірне подання режиму сну, що дозволяє батькам оцінити регулярність та тривалість нічного відпочинку дитини протягом тижня і своєчасно виявити порушення режиму сну.

ВИСНОВОК ДО РОЗДІЛУ 3

У третьому розділі розглянуто реалізацію кросплатформної клієнт-серверної системи агрегації та обробки медико-біологічних даних.

Спроековано та реалізовано архітектуру системи, що включає клієнтський застосунок на базі React Native та Expo з feature-based організацією коду і хмарний бекенд на платформі Supabase. Відокремлення шару представлення від шару даних та зосередження логіки авторизації на рівні СУБД забезпечують підтримуваність коду та унеможливлюють витік даних між обліковими записами незалежно від логіки клієнтського застосунку.

Реалізовано реляційну схему бази даних PostgreSQL з десятима таблицями відповідно до вимог третьої нормальної форми. Для кожної таблиці визначено RLS-політики, що ізолюють дані різних користувачів на рівні СУБД без додаткової логіки авторизації в клієнтському коді. Реалізовано тригер автоматичного призначення власника при створенні нового профілю дитини, що скорочує кількість клієнтських запитів із двох до одного.

Реалізовано систему автентифікації з підтримкою email/пароль та Google OAuth, а також синхронізацію між пристроями в реальному часі через Supabase Realtime із середньою затримкою 1–2 секунди.

На клієнтській частині реалізовано алгоритм класифікації антропометричних показників відповідно до стандартів ВООЗ з лінійною інтерполяцією перцентильних таблиць для точного вікового значення дитини, а також алгоритм аналізу структури сну з розподілом на денну та нічну складові. Розроблені алгоритми медично коректні та відповідають офіційним рекомендаціям ВООЗ щодо оцінювання фізичного розвитку дітей.

РОЗДІЛ 4

ТЕСТУВАННЯ ТА РОЗГОРТАННЯ СИСТЕМИ АГРЕГАЦІЇ МЕДИКО-БІОЛОГІЧНИХ ДАНИХ

4.1 Демонстрація роботи та функціональне тестування застосунку

Для підтвердження коректності роботи розробленого програмного продукту та наочної ілюстрації його основного функціоналу покроково розглянуто основні процеси взаємодії користувача з системою. Цей процес слугуватиме для функціонального тестування ключових аспектів застосунку та підтвердження відповідності реалізованих функцій поставленим вимогам.

4.1.1 Реєстрація та автентифікація користувача

При першому запуску застосунку користувач потрапляє на екран автентифікації, наведений на рисунку 4.1. Верхня частина екрана оформлена у вигляді брендової зони з фіолетовим градієнтним фоном та логотипом застосунку, що формує впізнаваний візуальний стиль. Нижня частина містить білу картку з двома варіантами входу до системи: формою email та пароля і кнопкою «Продовжити з Google». У правому верхньому куті розміщено перемикач мови інтерфейсу, що дозволяє обрати між українською та англійською мовами ще до входу в систему.

У разі обрання входу через Google застосунок ініціює OAuth 2.0 потік: системний браузер відкриває сторінку авторизації Google, де користувач надає дозвіл на передачу базових даних профілю. Після успішної авторизації застосунок автоматично отримує токени доступу та зберігає сесію на пристрої, що забезпечує збереження входу між запусками без необхідності

повторної автентифікації.

При вході через email та пароль облікові дані передаються на сервер через захищений канал HTTPS. Новим користувачам доступна реєстрація за посиланням «Немає акаунту? Зареєструватись», після якої система надсилає підтвердження на email. Після успішного підтвердження та входу застосунок розпізнає активну сесію та перенаправляє користувача до головного екрана зі вкладками.

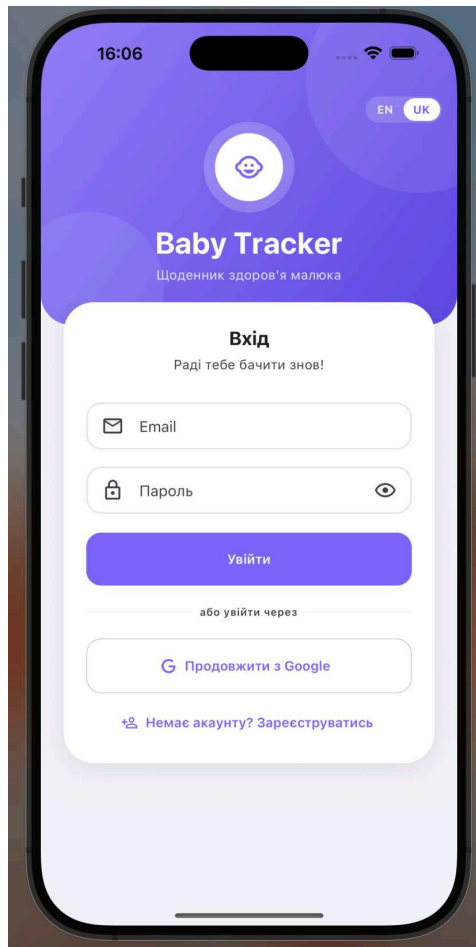


Рисунок 4.1 – Екран автентифікації застосунку

4.1.2 Додавання та відстеження активностей

Після входу в систему користувач потрапляє на головний екран застосунку, наведений на рисунку 4.2. У верхній частині відображається вітальна картка з іменем авторизованого користувача та поточною датою. Під нею розміщено картку активної дитини з аватаром, ім'ям та кількістю повних

місяців від дати народження; праворуч — перемикач між кількома дітьми у вигляді кружальців аватарів. Нижче знаходяться три лічильники поточного дня: кількість годувань, тривалість сну та кількість змін підгузків — вони оновлюються в реальному часі одразу після внесення кожного нового запису.

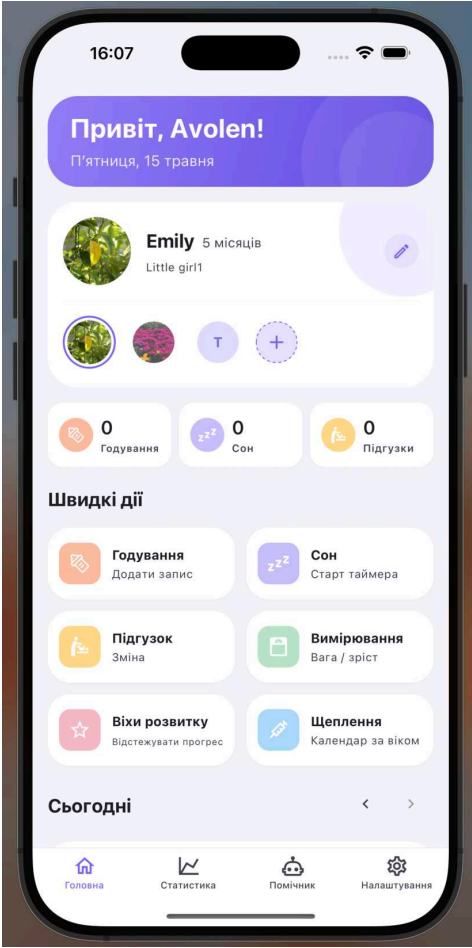


Рисунок 4.2 – Головний екран застосунку

Для внесення запису годування користувач натискає картку «Годування» у сітці швидких дій. Застосунок відкриває модальну форму, наведену на рисунку 4.3, де у верхній частині розміщена сітка з п'ятьма типами годування: ліва груди, права груди, пляшка з грудним молоком, пляшка з сумішшю та тверда їжа. Кожен варіант представлений карткою з кольоровою іконкою та текстовим підписом; активний тип підсвічується кольоровою рамкою. Після вибору типу користувач може натиснути «Старт зараз» для запуску таймера активного сеансу або вручну ввести тривалість

минулого годування у полі нижче та натиснути «Зберегти».

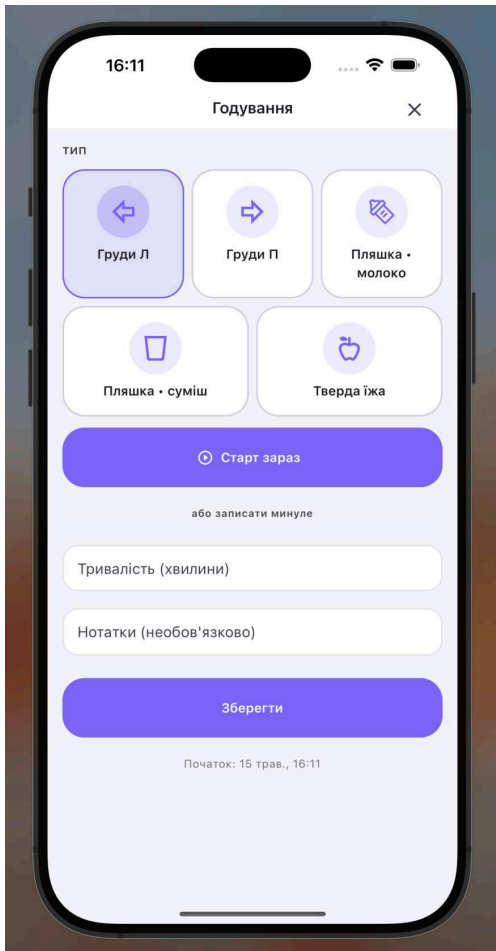


Рисунок 4.3 – Форма додавання запису годування

При натисканні «Старт зараз» застосунок фіксує початок сеансу, а на головному екрані з’являється таймер з відліком тривалості поточного сеансу. При натисканні «Зупинити» сеанс завершується і новий запис одразу з’являється в хронологічній стрічці подій поточного дня у нижній частині головного екрана.

Аналогічно реалізовано журналювання сну: натискання картки «Сон» відкриває форму з вибором типу (денний або нічний) та кнопкою запуску таймера. Активний сеанс сну відображається таймером на головному екрані і завершується натисканням «Зупинити», після чого запис автоматично з’являється у стрічці поточного дня. Зміни підгузків та одиничні події — досягнення й вакцинації — фіксуються відповідними картками у сітці

швидких дій без потреби у додаткових полях.

4.1.3 Моніторинг антропометричних показників

Для внесення вимірювання маси тіла, зросту або окружності голови користувач обирає картку «Вимірювання» на головному екрані. Відкривається форма з полями вибору типу показника, числового значення та одиниці вимірювання. Після збереження запис зберігається у базі даних і стає доступним у модулі статистики антропометрії. Екран вимірювань відображає окрему картку для кожного типу показника з графіком динаміки та позначенням перцентильних зон ВООЗ.

На рисунку 4.4 наведено екран антропометричного моніторингу з двома графіками: зросту та окружності голови. Для кожного показника застосунок виконує лінійну інтерполяцію між рядками перцентильних таблиць і обчислює п'ять граничних значень: P3, P15, P50, P85 та P97 для точного вікового значення дитини. Фон графіка розфарбовується відповідно до зони, в яку потрапляє значення: зелена зона між P15 та P85 відповідає нормі, жовті зони — граничним значенням P3–P15 та P85–P97, червоні зони — відхиленням нижче P3 та вище P97. Під кожним графіком відображається текстова класифікація останнього виміряного значення.

Під графіком кожного показника розміщена кнопка переходу до повної хронологічної історії вимірювань, наведеної на рисунку 4.5. Записи згруповані за датою у порядку спадання та містять тип показника, числове значення і точний час внесення. Такий підхід дозволяє батькам відстежувати довгострокову динаміку фізичного розвитку дитини і виявляти систематичні відхилення задовго до їх перетворення на клінічно значущі. У разі виявлення значень у червоній зоні (нижче P3 або вище P97) батькам рекомендується консультація педіатра для виключення патологічних причин відхилення.

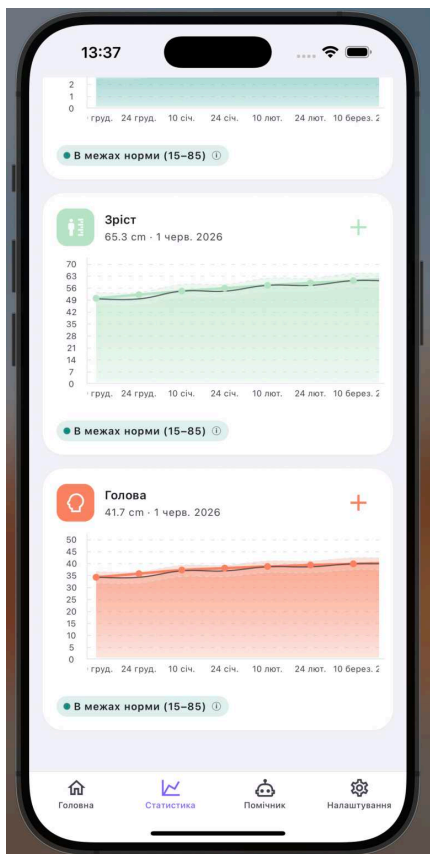


Рисунок 4.4 – Екран вимірювань із перцентильними зонами

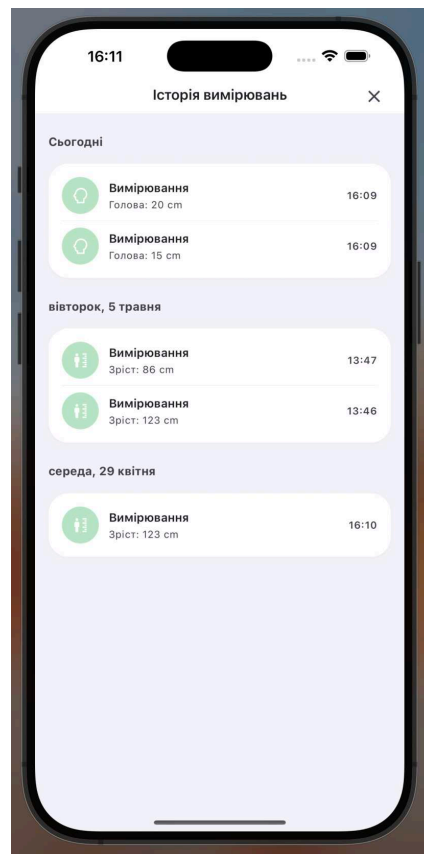


Рисунок 4.5 – Екран історії вимірювань

4.1.4 Аналітика активностей

Екран статистики, наведений на рисунку 4.6, відображає аналітику за останні 7 днів у вигляді трьох стовпчастих графіків: кількість годувань, загальна тривалість сну та кількість змін підгузків по днях тижня. Кожен стовпець відповідає одному дню, що дозволяє одразу побачити тенденції та відхилення від звичного режиму дитини.

Для глибшого аналізу режиму сну реалізовано окремий графік розподілу сну за часом доби: вісь Y відображає години від 00:00 до 24:00, а кожен стовпець показує діапазон активного сну у відповідний день, що дозволяє візуально виявляти порушення нічного режиму або надмірний денний сон. Під графіком сну відображається підсумковий рядок із загальною тривалістю денного та нічного сну за тиждень. Якщо середня

кількість змін підгузків виходить за межі норми, екран відображає відповідне попереджувальне повідомлення.

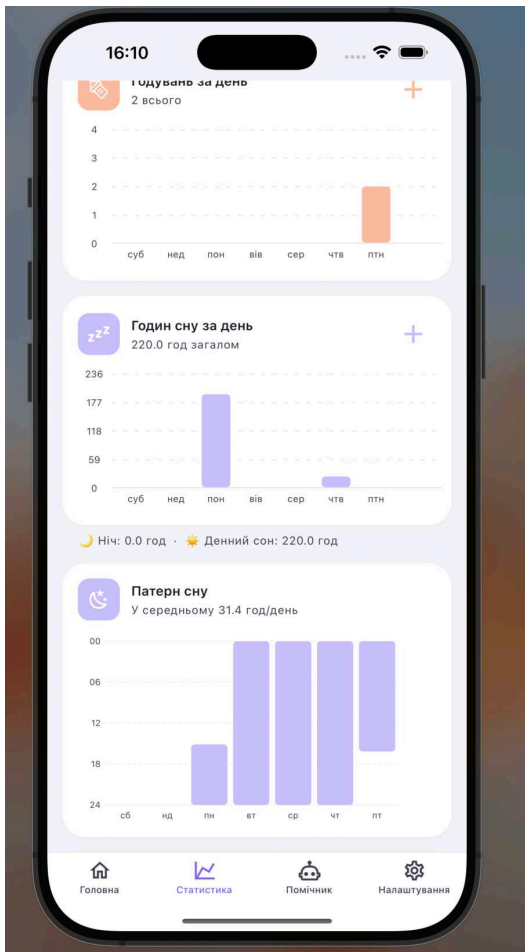


Рисунок 4.6 – Екран статистики за 7 днів

4.1.5 Параметри профілю та персоналізація інтерфейсу

Екран налаштувань, наведений на рисунку 4.7, організований у три тематичні секції. Секція «Профіль» надає доступ до редагування імені та адреси електронної пошти, а також до зміни пароля через відповідні модальні форми. Секція «Налаштування» містить перемикач мови інтерфейсу між українською та англійською, а також вибір теми оформлення; зміна мови набирає чинності миттєво без перезавантаження застосунку. Секція «Акаунт» включає кнопки виходу із системи та видалення облікового запису, кожна з яких потребує підтвердження у діалоговому вікні для запобігання випадковим деструктивним діям.

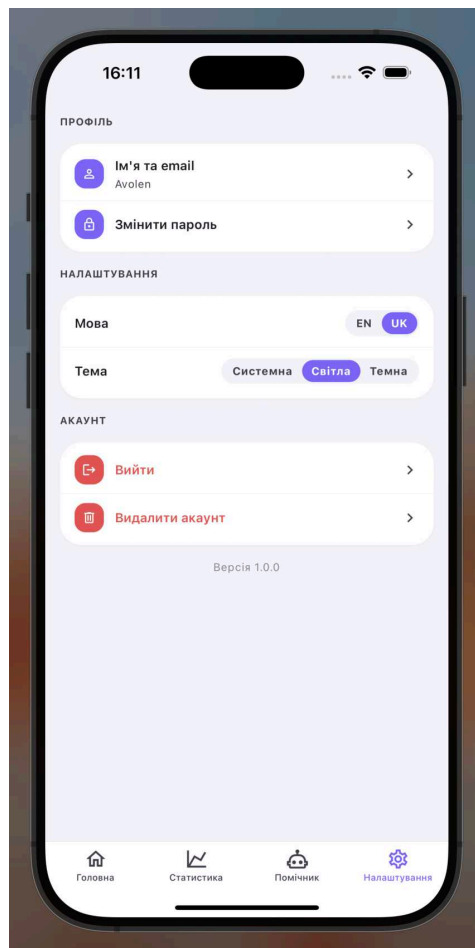


Рисунок 4.7 – Екран налаштувань застосунку

Після покрокової демонстрації основних сценаріїв взаємодії з системою розглянемо результати тестування її ключових функціональних модулів.

4.2 Тестування ключових функцій системи

4.2.1 Модульне тестування бізнес-логіки

Для верифікації коректності бізнес-логіки, що не залежить від компонентів UI, реалізовано набір модульних тестів з використанням фреймворку Jest у поєднанні з пресетом jest-expo та препроцесором ts-jest, що дозволяє виконувати TypeScript-тести безпосередньо в середовищі Node без попередньої компіляції.

Для тестування React-компонентів інтегровано бібліотеку `@testing-library/react-native`, яка надає утиліти для рендерингу компонентів у тестовому оточенні та перевірки їх поведінки за доступністю, а не за внутрішньою структурою.

Тести організовано у директорії `app/__tests__` із дзеркальною структурою відносно основного коду. Покрито шість функціональних областей:

- `features/measurements/who.test.ts` — тести алгоритму перцентильної класифікації: перевірка інтерполяції між суміжними місяцями, повернення `null` для віку понад 24 місяці, відмінність P50 між статями, коректність меж перцентильних зон;
- `lib/age.test.ts` — тести функції форматування віку дитини: коректне відображення у днях, тижнях, місяцях та роках залежно від вікового діапазону, обробка майбутніх дат народження;
- `lib/aggregate.test.ts` — тести агрегації активностей по днях: коректне формування 7-денних бакетів, підрахунок подій, підсумовування зважених значень;
- `features/milestones/labels.test.ts`,
`features/vaccinations/labels.test.ts` — тести відображення категорій досягнень та вакцинацій;
- `features/errors/translate.test.ts` — тести перекладу серверних помилок у зрозумілі повідомлення для користувача.

Оскільки тести виконуються в середовищі Node, React Native-модулі, що залежають від нативного коду, замінюються мінімальними заглушками через файл `__mocks__/react-native.js`. Такий підхід дозволяє запускати тести без емулятора або фізичного пристрою — виключно з командного рядка командою `jest`.

4.2.2 Перевірка безпеки та ізоляції даних

Коректність роботи RLS-політик є критично важливою вимогою для застосунку, що зберігає персональні медичні дані. Тестування проводилося з двома тестовими обліковими записами: обліковий запис А з дитиною Emily та набором записів активностей, і обліковий запис Б без жодних доданих дітей. Після входу в систему від імені облікового запису Б та виконання запиту на отримання списку дітей застосунок повернув порожній список, що підтверджує коректну роботу RLS-політики для таблиці children.

Наступним кроком перевірялася ізоляція записів активностей. За умов активної сесії облікового запису Б було сформовано прямий SQL-запит до таблиці feedings без фільтрів через клієнт Supabase. Незважаючи на відсутність умови WHERE child_id = ... у клієнтському коді, PostgREST автоматично застосував RLS-фільтр і повернув порожній набір рядків, оскільки обліковий запис Б не має жодного запису в таблиці caregivers. Це підтверджує, що захист даних реалізований не на рівні клієнтського застосунку, а на рівні СУБД, і не може бути обійдений навіть при наявності прямого доступу до API.

Додатково перевірялася коректність роботи тригера автоматичного призначення власника. При створенні нового профілю дитини від імені облікового запису Б у таблиці caregivers автоматично з'явився запис з парою (profile_id облікового запису Б, child_id нової дитини) та роллю owner. Після цього запити до feedings та інших таблиць активностей для цієї дитини почали повертати дані, що підтверджує коректну роботу механізму автоматичного призначення прав доступу без додаткових дій з боку клієнтського коду.

4.2.3 Тестування синхронізації між пристроями

Синхронізація між пристроями реалізована через Supabase Realtime та є однією з ключових конкурентних переваг системи порівняно з аналогами. Для тестування цієї функціональності застосунок було одночасно відкрито на двох пристроях — iPhone 14 Pro та iPad Air — під одним обліковим записом із активною дитиною Emily. Обидва пристрої підписалися на канал синхронізації для активної дитини, що відстежує зміни в усіх таблицях активностей.

При додаванні нового запису про годування на першому пристрої було зафіксовано час від збереження форми до появи нового запису у стрічці подій на другому пристрої. У трьох незалежних вимірюваннях цей час склав 1,2, 1,4 та 1,8 секунди відповідно, в середньому — близько 1,5 секунди. Такий час відгуку є цілком прийнятним для сценарію спільного ведення записів двома батьками і не потребує ручного оновлення сторінки. Аналогічний результат отримано при запуску та зупинці таймера сну: зміна стану активного сеансу миттєво відображалася на обох пристроях.

Механізм синхронізації коректно обробляв і конкурентні сценарії: при одночасному додаванні двох різних записів з обох пристроїв обидва записи з'явилися у стрічці подій на кожному пристрої без конфліктів або втрат даних. PostgreSQL забезпечив атомарність кожної вставки на рівні транзакцій, а Supabase Realtime доставив події оновлення незалежно для кожного запису. Таким чином, архітектурне рішення щодо використання Realtime-підписок у поєднанні з інвалідацією кешу TanStack Query продемонструвало надійну роботу в умовах паралельної взаємодії кількох клієнтів з одними даними.

4.3 Аналіз результатів тестування

Модульні тести повністю пройшли на всіх шести охоплених функціональних областях без помилок. За результатами проведеного функціонального тестування всі ключові сценарії взаємодії користувача з системою виконались коректно. Автентифікація через email та Google OAuth функціонує стабільно на обох платформах — iOS та Android. Журналювання всіх шести типів активностей (годування, сон, підгузок, вимірювання, досягнення, вакцинація) виконується без помилок, з правильним збереженням усіх полів у базі даних та коректним відображенням у хронологічній стрічці подій.

Алгоритм класифікації антропометричних показників за стандартами ВООЗ продемонстрував коректну роботу на тестових наборах даних. Значення зросту 86 см для дитини у 5 місяців правильно класифікується як «Вище 97-го перцентиля», оскільки це значення суттєво перевищує верхню межу норми для цього вікового діапазону. Значення окружності голови 20 см для тієї самої дитини коректно класифікується як «Нижче 3-го перцентиля». Обидві класифікації збігаються з розрахунковими значеннями перцентильних таблиць ВООЗ, підтверджуючи правильність реалізованого алгоритму лінійної інтерполяції.

Тестування безпеки підтвердило повну ізоляцію даних між обліковими записами на рівні СУБД. Синхронізація між пристроями в реальному часі продемонструвала затримку близько 1,5 секунди, що є прийнятним показником для даного класу застосунків.

4.4 Розгортання бекенду та публікація застосунку

Розгортання бекенду виконується через Supabase Dashboard. Спочатку створюється новий проєкт із вибором географічного регіону розміщення сервера. Після ініціалізації проєкту застосовується схема бази даних через виконання SQL-міграційних файлів у редакторі Dashboard: створюються всі десять предметних таблиць, визначаються RLS-політики та тригер призначення власника. Паралельно в розділі Auth активується провайдер Google OAuth із введенням Client ID та Client Secret, отриманих у Google Cloud Console, а також визначаються допустимі URL-адреси перенаправлення для deep-link застосунку.

Збірка та публікація клієнтського застосунку виконується через Expo Application Services. Конфігурація проєкту описана у файлі `app.json`, де визначаються ідентифікатор пакету (`com.babyjourney.app`), версія, дозволи та конфігурація глибоких посилань для OAuth. Змінні середовища — URL та анонімний ключ Supabase — зберігаються у файлі `.env` та передаються до збірки через механізм EAS Environment Variables. Команда `eas build --platform ios --profile production` ініціює хмарну збірку на серверах Expo; після її завершення артефакт `.ipa` готовий до відправки в App Store Connect. Аналогічна команда для платформи Android генерує `.aab`-файл для завантаження у Google Play Console.

Для автоматизації контролю якості коду налаштовано конвеєр безперервної інтеграції на базі GitHub Actions. При кожному push до гілки `main` та при створенні pull request автоматично запускається послідовність із трьох перевірок: статичний аналіз коду інструментом ESLint (`npx expo lint`), перевірка типів компілятором TypeScript у режимі `--noEmit` та виконання повного набору модульних тестів командою `npm test --ci`. Конвеєр виконується на хмарному агенті Ubuntu з Node.js версії 20 і

використовує кешування залежностей npm для скорочення часу виконання. Така автоматизація унеможливорює злиття коду з типовими помилками TypeScript або регресіями у тестах, забезпечуючи стабільність основної гілки репозиторію на всіх етапах розробки.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						66
Зм.	Лист	№ докум.	Підп.	Дата		

ВИСНОВОК ДО РОЗДІЛУ 4

У четвертому розділі проведено демонстрацію роботи та функціональне тестування розробленої кроссплатформної системи агрегації та обробки медико-біологічних даних.

Покроково розглянуто основні сценарії взаємодії користувача з системою: реєстрацію та автентифікацію через email/пароль та Google OAuth, журналювання активностей із підтримкою активних таймерів сну та годування, а також моніторинг антропометричних показників із класифікацією за стандартами ВООЗ. Усі розглянуті сценарії виконались коректно відповідно до очікуваної поведінки.

Верифіковано коректність бізнес-логіки за допомогою набору модульних тестів, що покривають алгоритм перцентильної класифікації, агрегацію активностей та форматування вікових показників. Підтверджено коректну ізоляцію даних між обліковими записами на рівні RLS-політик PostgreSQL: жоден із тест-кейсів не виявив витоку даних. Тестування синхронізації між двома пристроями підтвердило стабільну роботу Supabase Realtime із середньою затримкою 1,5 секунди при паралельній взаємодії кількох клієнтів.

Описано процес розгортання бекенду на платформі Supabase та збірки клієнтського застосунку через Expo Application Services для платформ iOS та Android, що завершує повний цикл розробки програмного продукту.

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		67

ВИСНОВКИ

У ході виконання дипломного проєкту розроблено кросплатформну клієнт-серверну систему агрегації та обробки медико-біологічних даних з використанням хмарних технологій — мобільний застосунок для відстеження розвитку немовляти на платформах iOS та Android.

Проведено аналіз предметної області та огляд трьох популярних аналогів (Sprout Baby Tracker, Huckleberry, Glow Baby). Встановлено, що жоден із них не поєднує в собі повноцінного антропометричного моніторингу відповідно до стандартів ВООЗ, синхронізації між пристроями в реальному часі.

Обрано технологічний стек: React Native та Expo для клієнтської частини, Supabase з PostgreSQL — для хмарного бекенду. Спроектовано реляційну схему бази даних із захистом на рівні рядків (RLS), що забезпечує ізоляцію даних між користувачами на рівні СУБД.

Реалізовано такі модулі системи: журнал активностей немовляти (годування, сон, підгузники), моніторинг антропометричних показників із кривими ВООЗ, управління вакцинаціями та досягненнями, а також синхронізація між пристроями в реальному часі.

Проведено тестування на рівнях юніт-тестів та функціонального тестування. Застосунок розгорнуто на хмарній інфраструктурі та готовий до використання.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						68
Зм.	Лист	№ докум.	Підп.	Дата		

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Sprout Baby: Tracker & Tips [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://apps.apple.com/app/sprout-baby/id524300416>.
2. Huckleberry — Baby & Toddler Sleep [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://huckleberrycare.com>.
3. Glow Baby Tracker [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://glowing.com/baby>.
4. React Native — Learn once, write anywhere [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://reactnative.dev>.
5. Expo — Build one JavaScript/TypeScript project that runs natively on all your users' devices [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://expo.dev>.
6. Expo Router — File-based routing for React Native [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://docs.expo.dev/router/introduction/>.
7. TypeScript — JavaScript with syntax for types [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://www.typescriptlang.org>.
8. TanStack Query (React Query) — Asynchronous State Management [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://tanstack.com/query>.
9. Zustand — Bear necessities for state management in React [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://zustand-demo.pmnd.rs>.
10. i18next — Internationalization framework for JavaScript [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://www.i18next.com>.
11. react-i18next — Internationalization for React [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://react.i18next.com>.

12. React Native Paper — Material Design for React Native [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://reactnativepaper.com>.
13. React Native Reanimated — Smooth animations and interactions in React Native [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://docs.swmansion.com/react-native-reanimated>.
14. react-native-gifted-charts — Charts for React Native [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://github.com/Abhinandan-Kushwaha/react-native-gifted-charts>.
15. Expo SecureStore — Encrypted key-value storage for Expo [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://docs.expo.dev/sdk/securestore>.
16. Expo Application Services (EAS) — Build, Submit, and Update React Native apps [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://expo.dev/eas>.
17. Supabase — The Open Source Firebase Alternative [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://supabase.com>.
18. PostgreSQL: The World's Most Advanced Open Source Relational Database [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://www.postgresql.org>.
19. Supabase Row Level Security — Secure your data using Postgres policies [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://supabase.com/docs/guides/auth/row-level-security>.
20. Supabase Auth — User Management for your application [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://supabase.com/docs/guides/auth>.

21. Supabase Realtime — Listen to database changes via websockets [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://supabase.com/docs/guides/realtime>.
22. Supabase Storage — Store and serve large files [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://supabase.com/docs/guides/storage>.
23. Supabase Edge Functions — Globally distributed TypeScript functions [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://supabase.com/docs/guides/functions>.
24. Supabase vs Firebase: An Honest Comparison [Електронний ресурс]. — 2024. — Режим доступу до ресурсу: <https://supabase.com/alternatives/supabase-vs-firebase>.
25. Deno — A modern runtime for JavaScript and TypeScript [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://deno.com>.
26. Groq — Fast AI Inference Platform [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://groq.com>.
27. pgvector: Open-source vector similarity search for Postgres [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://github.com/pgvector/pgvector>.
28. pgvector: Embeddings and vector similarity in PostgreSQL [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://supabase.com/docs/guides/ai/vector-columns>.
29. LangChain.js — Build context-aware reasoning applications [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://docs.langchain.com>.
30. Llama 3 — Meta’s open-source large language model [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://www.llama.com>.

31. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis [та ін.] // Advances in Neural Information Processing Systems. — 2020. — Т. 33. — С. 9459—9474.
32. WHO Child Growth Standards [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://www.who.int/tools/child-growth-standards>.
33. *World Health Organization*. WHO Child Growth Standards: Length/height-for-age, weight-for-age, weight-for-length, weight-for-height and body mass index-for-age / World Health Organization // WHO Press. — 2006.
34. Development of a WHO growth reference for school-aged children and adolescents / M. de Onis [та ін.] // Bulletin of the World Health Organization. — 2007. — Т. 85, № 9. — С. 660—667.
35. OWASP Mobile Application Security Verification Standard (MASVS) [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://mas.owasp.org/MASVS>.
36. freeRASP — Mobile In-App Protection [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://github.com/talsec/Free-RASP-ReactNative>.
37. Sentry — Application Performance Monitoring & Error Tracking [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://sentry.io>.
38. Jest — Delightful JavaScript Testing [Електронний ресурс]. — 2026. — Режим доступу до ресурсу: <https://jestjs.io>.

ДОДАТОК А

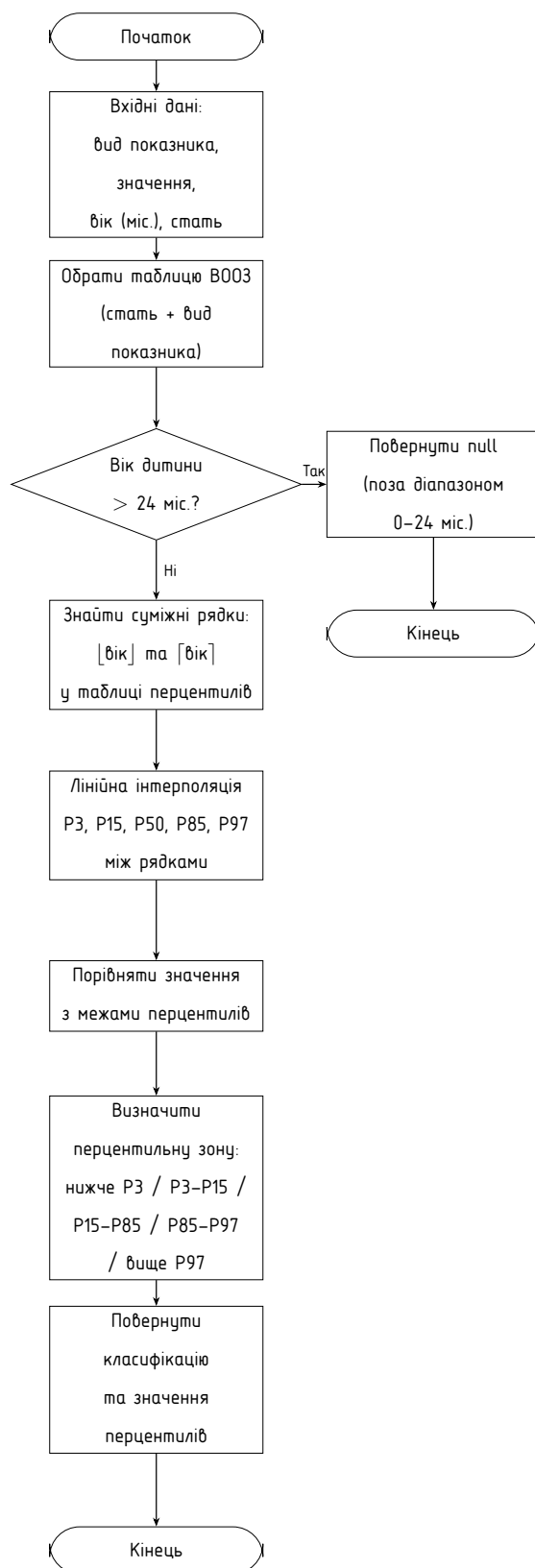
Кросплатформна клієнт-серверна система агрегації та обробки
медико-біологічних даних з використанням хмарних технологій

Принципова схема алгоритму перцентильної класифікації антропометричних показників

ІАЛЦ.467200.004 Д1

Аркушів 1

Київ 2026 р



					ІАЛЦ.467200.004 Д1			
Зм.	Лист	№ докум.	Підп.	Дата				
Розробив	Льоскін І. В.				Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій Принципова схема	Лит.	Аркуш	Аркушів
Перевірів	Кобилюк А. Г.						1	1
Н. контр.	Нікольський С.С.					НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ ІО-25		
Затвердив	Волокита А. М.							

ДОДАТОК Б

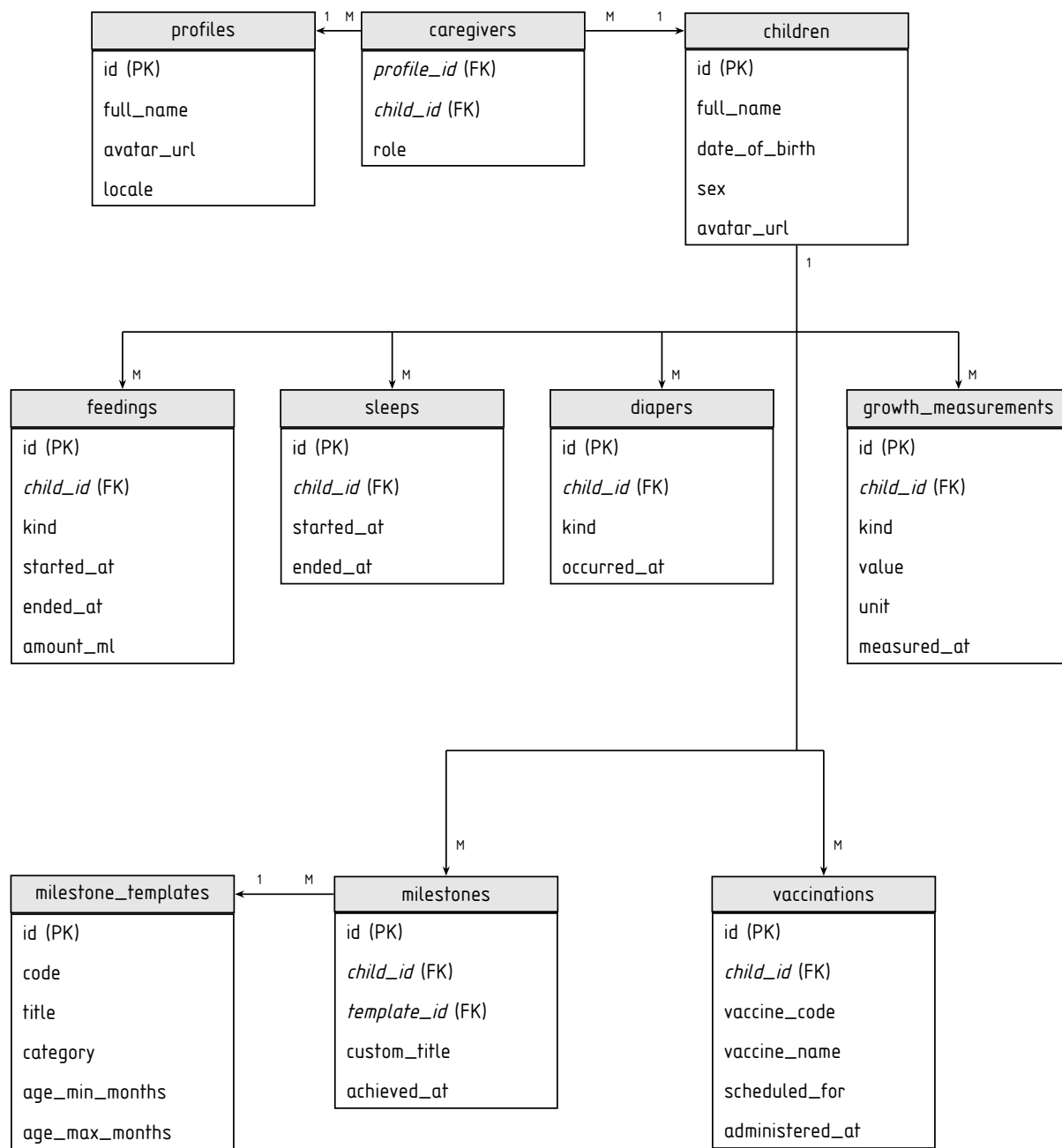
Кросплатформна клієнт-серверна система агрегації та обробки
медико-біологічних даних з використанням хмарних технологій

Функціональна схема бази даних системи

ІАЛЦ.467200.005 Д2

Аркушів 1

Київ 2026 р



					ІАЛЦ.467200.005 Д2			
Зм.	Лист	№ докум.	Підп.	Дата				
Розробив	Льоскін І. В.							
Перевірів	Кобилюк А. Г.				Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій		Лит.	Аркуш
							1	1
Н. контр.	Нікольський С.С.				Функціональна схема		НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ ІО-25	
Затвердив	Волокита А. М.							

ДОДАТОК В

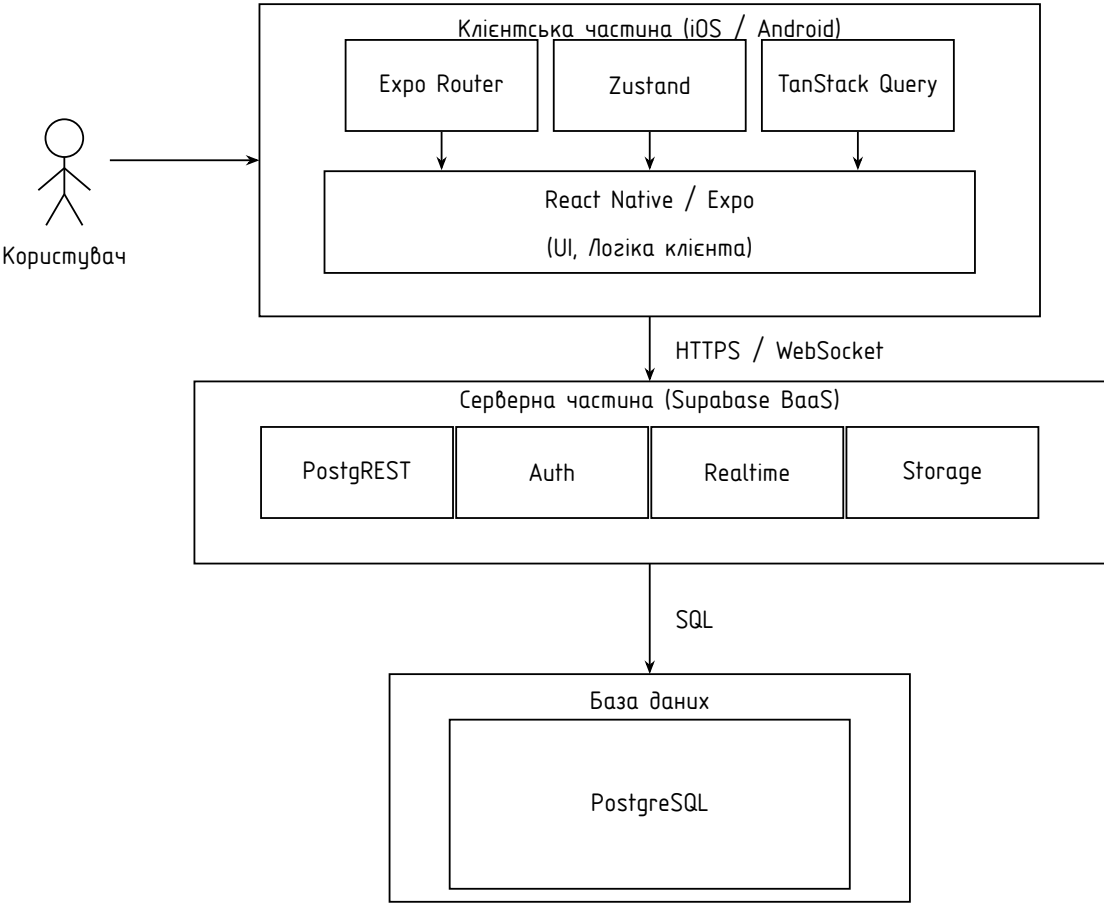
Кросплатформна клієнт-серверна система агрегації та обробки
медико-біологічних даних з використанням хмарних технологій

Структурна схема клієнт-серверної системи

ІАЛЦ.467200.006 ДЗ

Аркушів 1

Київ 2026 р



					ІАЛЦ.467200.006 ДЗ			
Зм.	Лист	№ докум.	Підп.	Дата	Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій Структурна схема	Лит.	Аркуш	Аркушів
Розробив	Льоскін І. В.						1	1
Перевірів	Кобилюк А. Г.							
Н. контр.	Нікольський С.С.					НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ ІО-25		
Затвердив	Волокита А. М.							

ДОДАТОК Г

Кросплатформна клієнт-серверна система агрегації та обробки
медико-біологічних даних з використанням хмарних технологій

Текст програмного коду

ІАЛЦ.467200.007 Д4

Аркушів 21

Київ 2026 р

```

-- supabase/migrations/20260423132703_initial_schema.sql
-- =====
-- Initial schema: core domain for a baby health tracker
-- =====

create extension if not exists "pgcrypto";

create type caregiver_role as enum (
    'owner', 'co_parent', 'caregiver', 'pediatrician');

create type feeding_kind as enum (
    'breast_left', 'breast_right',
    'bottle_breast_milk', 'bottle_formula', 'solid');

create type diaper_kind as enum ('wet', 'dirty', 'mixed', 'dry');

create type measurement_kind as enum (
    'weight', 'height', 'head_circumference');

-- Profiles - one row per auth user
create table public.profiles (
    id          uuid primary key references auth.users(id) on delete cascade,
    full_name   text,
    avatar_url  text,
    locale      text default 'uk',
    created_at  timestampz not null default now(),
    updated_at  timestampz not null default now()
);

create or replace function public.handle_new_user()
returns trigger language plpgsql security definer
set search_path = public as $$
begin
    insert into public.profiles (id, full_name)
    values (new.id, coalesce(new.raw_user_meta_data->>'full_name', ''));
    return new;
end;
$$;

create trigger on_auth_user_created
    after insert on auth.users
    for each row execute function public.handle_new_user();

```

					ІАЛЦ.467200.007 Д4			
Зм.	Лист	№ докум.	Підп.	Дата	Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій Текст програмного коду	Лит.	Аркуш	Аркушів
Розробив	Льоскін І. В.						1	21
Перевірив	Кобиліук А. Г.							
Н. контр.	Нікольський С.С.					НТУУ "КПІ ім. Ізгоря Сікорського", ФІОТ ІО-25		
Затвердив	Волокита А. М.							

```

-- Children
create table public.children (
    id            uuid primary key default gen_random_uuid(),
    full_name     text not null,
    date_of_birth date not null,
    sex           text check (sex in ('male', 'female', 'unspecified'))
                  default 'unspecified',
    avatar_url    text,
    notes         text,
    created_at    timestampz not null default now(),
    updated_at    timestampz not null default now()
);

-- Caregivers - M:N between profiles and children, with roles

create table public.caregivers (
    child_id      uuid not null references public.children(id) on delete cascade,
    profile_id    uuid not null references public.profiles(id) on delete cascade,
    role          caregiver_role not null default 'co_parent',
    created_at    timestampz not null default now(),
    primary key (child_id, profile_id)
);

create index caregivers_profile_idx on public.caregivers(profile_id);

create or replace function public.is_caregiver(p_child uuid)
returns boolean language sql stable security definer
set search_path = public as $$
    select exists (
        select 1 from public.caregivers
        where child_id = p_child and profile_id = auth.uid()
    );
$$;

-- Feedings
create table public.feedings (
    id            uuid primary key default gen_random_uuid(),
    child_id      uuid not null references public.children(id) on delete cascade,
    kind          feeding_kind not null,
    started_at    timestampz not null,
    ended_at      timestampz,
    amount_ml     numeric(6,1),
    notes         text,
    created_by    uuid references public.profiles(id),
    created_at    timestampz not null default now()
);

```

					ІАЛЦ.467200.007 Д4	Аркуш
						2
Зм.	Лист	№ докум.	Підп.	Дата		

```

create index feedings_child_started_idx
    on public.feedings(child_id, started_at desc);

-- Sleeps
create table public.sleeps (
    id          uuid primary key default gen_random_uuid(),
    child_id    uuid not null references public.children(id) on delete cascade,
    started_at  timestampz not null,
    ended_at    timestampz,
    notes       text,
    created_by  uuid references public.profiles(id),
    created_at  timestampz not null default now()
);
create index sleeps_child_started_idx
    on public.sleeps(child_id, started_at desc);

-- Diapers
create table public.diapers (
    id          uuid primary key default gen_random_uuid(),
    child_id    uuid not null references public.children(id) on delete cascade,
    kind        diaper_kind not null,
    occurred_at timestampz not null default now(),
    notes       text,
    created_by  uuid references public.profiles(id),
    created_at  timestampz not null default now()
);
create index diapers_child_occurred_idx
    on public.diapers(child_id, occurred_at desc);

-- Growth measurements
create table public.growth_measurements (
    id          uuid primary key default gen_random_uuid(),
    child_id    uuid not null references public.children(id) on delete cascade,
    kind        measurement_kind not null,
    value       numeric(7,2) not null,
    unit        text not null,
    measured_at timestampz not null default now(),
    notes       text,
    created_by  uuid references public.profiles(id),
    created_at  timestampz not null default now()
);
create index growth_child_measured_idx
    on public.growth_measurements(child_id, kind, measured_at desc);

-- Milestone templates (seeded once)
create table public.milestone_templates (

```

```

    id                uuid primary key default gen_random_uuid(),
    code              text unique not null,
    title             text not null,
    description        text,
    category           text not null,
    age_min_months     int not null,
    age_max_months     int not null
);

-- Milestones (per-child achievements)
create table public.milestones (
    id                uuid primary key default gen_random_uuid(),
    child_id          uuid not null references public.children(id) on delete cascade,
    template_id        uuid references public.milestone_templates(id),
    custom_title       text,
    achieved_at        date,
    notes             text,
    created_by         uuid references public.profiles(id),
    created_at         timestampz not null default now(),
    check (template_id is not null or custom_title is not null)
);

create index milestones_child_idx on public.milestones(child_id);

-- Vaccinations
create table public.vaccinations (
    id                uuid primary key default gen_random_uuid(),
    child_id          uuid not null references public.children(id) on delete cascade,
    vaccine_code       text not null,
    vaccine_name       text not null,
    scheduled_for      date,
    administered_at    date,
    dose_number        int,
    notes             text,
    created_by         uuid references public.profiles(id),
    created_at         timestampz not null default now()
);

create index vaccinations_child_idx on public.vaccinations(child_id);

-- Row Level Security
alter table public.profiles          enable row level security;
alter table public.children          enable row level security;
alter table public.caregivers        enable row level security;
alter table public.feedings          enable row level security;
alter table public.sleeps            enable row level security;
alter table public.diapers           enable row level security;
alter table public.growth_measurements enable row level security;

```

					ІАЛЦ.467200.007 Д4	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		4

```

alter table public.milestone_templates enable row level security;
alter table public.milestones          enable row level security;
alter table public.vaccinations         enable row level security;

create policy "profiles_self_select" on public.profiles
  for select using (id = auth.uid());
create policy "profiles_self_update" on public.profiles
  for update using (id = auth.uid());

create policy "children_caregiver_select" on public.children
  for select using (public.is_caregiver(id));
create policy "children_authenticated_insert" on public.children
  for insert with check (auth.uid() is not null);
create policy "children_caregiver_update" on public.children
  for update using (public.is_caregiver(id));

create policy "caregivers_caregiver_select" on public.caregivers
  for select using (public.is_caregiver(child_id));

create policy "feedings_caregiver_select" on public.feedings
  for select using (public.is_caregiver(child_id));
create policy "feedings_caregiver_insert" on public.feedings
  for insert with check (public.is_caregiver(child_id));
create policy "feedings_caregiver_update" on public.feedings
  for update using (public.is_caregiver(child_id));
create policy "feedings_caregiver_delete" on public.feedings
  for delete using (public.is_caregiver(child_id));

-- supabase/migrations/20260425132539_child_owner_trigger.sql
-- Auto-add the creator as the owner caregiver of every new child.
-- Lets the client do a single INSERT into children; Postgres handles
-- the caregivers row atomically inside a SECURITY DEFINER trigger.

create or replace function public.add_child_owner()
returns trigger language plpgsql security definer
set search_path = public as $$
begin
  if auth.uid() is not null then
    insert into public.caregivers (child_id, profile_id, role)
    values (new.id, auth.uid(), 'owner')
    on conflict (child_id, profile_id) do nothing;
  end if;
  return new;
end;
$$;

drop trigger if exists on_child_created on public.children;

```

					ІАЛЦ.467200.007 Д4	Аркуш
						5
Зм.	Лист	№ докум.	Підп.	Дата		


```

create trigger on_child_created
  after insert on public.children
  for each row execute function public.add_child_owner();

-- supabase/migrations/20260513200000_articles_pgvector.sql
-- Enable pgvector for semantic search.
-- gte-small produces 384-dimensional embeddings via Supabase AI.
create extension if not exists vector;

create table public.articles (
  id          uuid primary key default gen_random_uuid(),
  title       text not null,
  body        text not null,
  category    text not null,
  age_min_months int not null default 0,
  age_max_months int not null default 36,
  embedding    vector(384),
  created_at  timestamptz not null default now()
);

-- IVFFlat index for approximate nearest-neighbour search.
-- 20 lists is appropriate for <1 000 rows; increase for larger corpora.
create index articles_embedding_idx on public.articles
  using ivfflat (embedding vector_cosine_ops) with (lists = 20);

alter table public.articles enable row level security;
create policy "articles_authenticated_read" on public.articles
  for select using (auth.uid() is not null);

-- Semantic search: returns rows ranked by cosine similarity to the
-- query embedding, optionally filtered by child age.
create or replace function public.match_articles(
  query_embedding vector(384),
  match_count      int      default 3,
  p_age_months     int      default null
)
returns table (
  id          uuid,
  title       text,
  body        text,
  category    text,
  similarity float
)
language sql stable
as $$
  select id, title, body, category,
    1 - (embedding <=> query_embedding) as similarity

```

					ІАЛЦ.467200.007 Д4	Аркуш
						6
Зм.	Лист	№ докум.	Підп.	Дата		

```

from public.articles
where embedding is not null
  and (
    p_age_months is null
    or (age_min_months <= p_age_months and age_max_months >= p_age_months)
  )
order by embedding <=> query_embedding
limit match_count;
$$;

// app/types/domain.ts
/**
 * Domain types - re-exports derived from the auto-generated Supabase schema.
 * App code should import from here instead of `lib/database.types` directly.
 */
import type { Database } from '@lib/database.types';

type Tables = Database['public']['Tables'];
type Enums = Database['public']['Enums'];

export type Child = Tables['children']['Row'];
export type ChildInsert = Tables['children']['Insert'];
export type ChildUpdate = Tables['children']['Update'];
export type Sex = 'male' | 'female' | 'unspecified';

export type Caregiver = Tables['caregivers']['Row'];
export type CaregiverRole = Enums['caregiver_role'];

export type Feeding = Tables['feedings']['Row'];
export type FeedingInsert = Tables['feedings']['Insert'];
export type FeedingKind = Enums['feeding_kind'];

export type Sleep = Tables['sleeps']['Row'];
export type SleepInsert = Tables['sleeps']['Insert'];

export type Diaper = Tables['diapers']['Row'];
export type DiaperInsert = Tables['diapers']['Insert'];
export type DiaperKind = Enums['diaper_kind'];

export type GrowthMeasurement = Tables['growth_measurements']['Row'];
export type GrowthMeasurementInsert = Tables['growth_measurements']['Insert'];
export type MeasurementKind = Enums['measurement_kind'];

export type Milestone = Tables['milestones']['Row'];
export type MilestoneTemplate = Tables['milestone_templates']['Row'];

export type Vaccination = Tables['vaccinations']['Row'];

```

					ІАЛЦ.467200.007 Д4	Аркуш
						7
Зм.	Лист	№ докум.	Підп.	Дата		

```

export type VaccinationInsert = Tables['vaccinations']['Insert'];
export type VaccinationTemplate = Tables['vaccination_templates']['Row'];

// app/lib/supabase.ts
import 'react-native-url-polyfill/auto';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { createClient } from '@supabase/supabase-js';
import { AppState } from 'react-native';
import type { Database } from '../database.types';
import { env } from '../env';

export const supabase = createClient<Database>(env.supabaseUrl, env.supabaseAnonKey, {
  auth: {
    storage: AsyncStorage,
    autoRefreshToken: true,
    persistSession: true,
    detectSessionInUrl: false,
  },
});

AppState.addListener('change', (state) => {
  if (state === 'active') {
    supabase.auth.startAutoRefresh();
  } else {
    supabase.auth.stopAutoRefresh();
  }
});

/**
 * Reads the persisted session and returns the current user's id, or `null`
 * when no session exists. Used by every domain mutation to stamp
 * `created_by` on inserts.
 */
export async function getCurrentUserId(): Promise<string | null> {
  const { data } = await supabase.auth.getSession();
  return data.session?.user.id ?? null;
}

// app/features/measurements/who/index.ts
import { differenceInMonths, parseISO } from 'date-fns';

import {
  WHO_STANDARDS,
  type WhoMetric,
  type WhoRow,
  type WhoSex,
} from '../standards';

```

					ІАЛЦ.467200.007 Д4	Аркуш
						8
Зм.	Лист	№ докум.	Підп.	Дата		

```

import type { GrowthMeasurement, MeasurementKind, Sex } from '@types/domain';

const KIND_TO_METRIC: Record<MeasurementKind, WhoMetric> = {
  weight: 'weight',
  height: 'height',
  head_circumference: 'head_circumference',
};

function whoSex(sex: Sex | string | null | undefined): WhoSex {
  return sex === 'female' ? 'female' : 'male';
}

export type Bands = {
  p3: number;
  p15: number;
  p50: number;
  p85: number;
  p97: number;
};

const linterp = (a: number, b: number, t: number) => a + (b - a) * t;

function lookup(rows: readonly WhoRow[], ageMonths: number): Bands | null {
  if (ageMonths < rows[0][0] || ageMonths > rows[rows.length - 1][0])
    return null;
  const upperIdx = rows.findIndex((r) => r[0] >= ageMonths);
  if (upperIdx === -1) return null;
  const upper = rows[upperIdx];
  if (upper[0] === ageMonths || upperIdx === 0) {
    return {
      p3: upper[1], p15: upper[2], p50: upper[3],
      p85: upper[4], p97: upper[5],
    };
  }
  const lower = rows[upperIdx - 1];
  const t = (ageMonths - lower[0]) / (upper[0] - lower[0]);
  return {
    p3: linterp(lower[1], upper[1], t),
    p15: linterp(lower[2], upper[2], t),
    p50: linterp(lower[3], upper[3], t),
    p85: linterp(lower[4], upper[4], t),
    p97: linterp(lower[5], upper[5], t),
  };
}

export function whoBands(

```

```

    sex: Sex | string | null | undefined,
    kind: MeasurementKind,
    ageMonths: number,
  ): Bands | null {
    return lookup(
      WHO_STANDARDS[whoSex(sex)][KIND_TO_METRIC[kind]],
      ageMonths,
    );
  }

export function ageMonthsAt(dobIso: string, atIso: string): number {
  return Math.max(differenceInMonths(parseISO(atIso), parseISO(dobIso)), 0);
}

export type PercentileBand =
  | 'below_p3' | 'p3_p15' | 'p15_p85' | 'p85_p97' | 'above_p97';

export function classifyValue(value: number, bands: Bands): PercentileBand {
  if (value < bands.p3) return 'below_p3';
  if (value < bands.p15) return 'p3_p15';
  if (value <= bands.p85) return 'p15_p85';
  if (value <= bands.p97) return 'p85_p97';
  return 'above_p97';
}

export function classifyMeasurement(
  measurement: GrowthMeasurement,
  dobIso: string,
  sex: Sex | string | null | undefined,
): { bands: Bands; band: PercentileBand } | null {
  const months = ageMonthsAt(dobIso, measurement.measured_at);
  const bands = whoBands(sex, measurement.kind, months);
  if (!bands) return null;
  return { bands, band: classifyValue(measurement.value, bands) };
}

// app/hooks/use-sleep-pattern.ts
import { useMemo } from 'react';
import { addDays, format, parseISO, startOfDay } from 'date-fns';

import { useSleepsInRange } from '@features/sleeps/queries';
import { useLastDaysRange } from '@hooks/use-last-days-range';
import type { Sleep } from '@types/domain';

/** A single block within one day, hours expressed as floats in [0, 24]. */
export type SleepBlock = { startHour: number; endHour: number };

```

					ИАЛЦ.467200.007 Д4	Аркуш
						10
Зм.	Лист	№ докум.	Підп.	Дата		

```

export type SleepPatternDay = {
  /** ISO day key (yyyy-MM-dd). */
  key: string;
  /** Calendar date for axis labelling. */
  date: Date;
  blocks: SleepBlock[];
};

const HOURS_IN_DAY = 24;
const MS_IN_HOUR = 60 * 60 * 1000;

function splitSleepAtMidnight(
  sleep: Sleep,
  windowStart: Date,
  windowEnd: Date,
): { dayKey: string; block: SleepBlock }[] {
  const start = parseISO(sleep.started_at);
  const end = sleep.ended_at ? parseISO(sleep.ended_at) : new Date();

  // Clip the sleep to the visible window so a sleep that started weeks ago
  // and only finished today still produces just one block.
  const clipStart = start < windowStart ? windowStart : start;
  const clipEnd = end > windowEnd ? windowEnd : end;
  if (clipEnd <= clipStart) return [];

  const out: { dayKey: string; block: SleepBlock }[] = [];
  let cursor = clipStart;
  while (cursor < clipEnd) {
    const dayStart = startOfDay(cursor);
    const nextDayStart = addDays(dayStart, 1);
    const segmentEnd = clipEnd < nextDayStart ? clipEnd : nextDayStart;
    out.push({
      dayKey: format(dayStart, 'yyyy-MM-dd'),
      block: {
        startHour: (cursor.getTime() - dayStart.getTime()) / MS_IN_HOUR,
        endHour: (segmentEnd.getTime() - dayStart.getTime()) / MS_IN_HOUR,
      },
    });
    cursor = segmentEnd;
  }
  return out;
}

/**
 * Buckets the last `days` days of sleep entries for the active child into a
 * per-day list of intra-day blocks suitable for a 24-hour pattern chart.

```

```

* Sleeps that span midnight are split into two adjacent blocks; an
* unfinished sleep is rendered up to the current moment.
*/
export function useSleepPattern(childId: string | null, days: number): SleepPatternDay[] {
  const { from, to } = useLastDaysRange(days);
  const { data: sleeps = [] } = useSleepsInRange(childId, from, to);

  return useMemo(() => {
    const today = startOfDay(new Date());
    const buckets: SleepPatternDay[] = [];
    for (let i = days - 1; i >= 0; i--) {
      const date = addDays(today, -i);
      buckets.push({ key: format(date, 'yyyy-MM-dd'), date, blocks: [] });
    }
    const byKey = new Map(buckets.map((b) => [b.key, b]));

    const windowStart = buckets[0].date;
    const windowEnd = addDays(today, 1);

    for (const sleep of sleeps) {
      for (const { dayKey, block } of splitSleepAtMidnight(sleep, windowStart, windowEnd)) {
        const bucket = byKey.get(dayKey);
        if (bucket) bucket.blocks.push(block);
      }
    }
    return buckets;
  }, [sleeps, days]);
}

export const SLEEP_PATTERN_HOURS = HOURS_IN_DAY;

// app/lib/realtime.ts
import { useEffect } from 'react';
import { useQueryClient } from '@tanstack/react-query';

import { supabase } from '../supabase';

const TABLE_TO_KEY: Record<string, string> = {
  feedings: 'feedings',
  sleeps: 'sleeps',
  diapers: 'diapers',
  growth_measurements: 'measurements',
  milestones: 'milestones',
  vaccinations: 'vaccinations',
};

export function useChildRealtime(childId: string | null) {

```

					ІАЛЦ.467200.007 Д4	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		12

```

const qc = useQueryClient();

useEffect(() => {
  if (!childId) return;

  const channel = supabase.channel(`child:${childId}`);

  for (const table of Object.keys(TABLE_TO_KEY)) {
    channel.on(
      'postgres_changes',
      {
        event: '*',
        schema: 'public',
        table,
        filter: `child_id=eq.${childId}`,
      },
      () => {
        qc.invalidateQueries({
          queryKey: [TABLE_TO_KEY[table], childId],
        });
      },
    );
  }

  channel.subscribe();

  return () => {
    supabase.removeChannel(channel);
  };
}, [childId, qc]);
}

// app/features/feedings/queries.ts
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query';
import { startOfDay, endOfDay, format } from 'date-fns';

import { getCurrentUserId, supabase } from '@lib/supabase';
import type { Feeding, FeedingInsert, FeedingKind } from '@types/domain';

export const feedingsKey = (childId: string) =>
  ['feedings', childId] as const;
export const feedingsForDayKey = (childId: string, day: string) =>
  ['feedings', childId, 'day', day] as const;
export const feedingsInRangeKey =
  (childId: string, from: string, to: string) =>
    ['feedings', childId, 'range', from, to] as const;
export const activeFeedingKey = (childId: string) =>

```

					ІАЛЦ.467200.007 Д4	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		13


```

    ['feedings', childId, 'active'] as const;

export function useFeedingsInRange(
  childId: string | null,
  from: Date,
  to: Date,
) {
  const fromKey = format(from, 'yyyy-MM-dd');
  const toKey = format(to, 'yyyy-MM-dd');
  return useQuery({
    queryKey: childId
      ? feedingsInRangeKey(childId, fromKey, toKey)
      : ['feedings', 'noop-range'],
    enabled: Boolean(childId),
    queryFn: async (): Promise<Feeding[]> => {
      if (!childId) return [];
      const { data, error } = await supabase
        .from('feedings')
        .select('*')
        .eq('child_id', childId)
        .gte('started_at', startOfDay(from).toISOString())
        .lte('started_at', endOfDay(to).toISOString())
        .order('started_at', { ascending: true });
      if (error) throw error;
      return data;
    },
  });
}

export function useFeedingsForDay(childId: string | null, day: Date) {
  const dayKey = format(day, 'yyyy-MM-dd');
  return useQuery({
    queryKey: childId
      ? feedingsForDayKey(childId, dayKey)
      : ['feedings', 'noop'],
    enabled: Boolean(childId),
    queryFn: async (): Promise<Feeding[]> => {
      if (!childId) return [];
      const { data, error } = await supabase
        .from('feedings')
        .select('*')
        .eq('child_id', childId)
        .gte('started_at', startOfDay(day).toISOString())
        .lte('started_at', endOfDay(day).toISOString())
        .order('started_at', { ascending: false });
      if (error) throw error;
    },
  });
}

```

					ІАЛЦ.467200.007 Д4	Аркуш
						14
Зм.	Лист	№ докум.	Підп.	Дата		

```

        return data;
    },
});
}

export function useActiveFeeding(childId: string | null) {
    return useQuery({
        queryKey: childId
            ? activeFeedingKey(childId)
            : ['feedings', 'noop-active'],
        enabled: Boolean(childId),
        queryFn: async (): Promise<Feeding | null> => {
            if (!childId) return null;
            const { data, error } = await supabase
                .from('feedings')
                .select('*')
                .eq('child_id', childId)
                .in('kind', ['breast_left', 'breast_right'])
                .is('ended_at', null)
                .order('started_at', { ascending: false })
                .limit(1)
                .maybeSingle();
            if (error) throw error;
            return data;
        },
    });
}

export function useStartFeeding() {
    const qc = useQueryClient();
    return useMutation({
        mutationFn: async ({
            childId,
            kind,
        }): {
            childId: string;
            kind: FeedingKind;
        }): Promise<Feeding> => {
            const userId = await getCurrentUserId();
            const { data, error } = await supabase
                .from('feedings')
                .insert({
                    child_id: childId,
                    kind,
                    started_at: new Date().toISOString(),
                    ended_at: null,

```

```

        created_by: userId,
      })
      .select('*')
      .single();
      if (error) throw error;
      return data;
    },
    onSuccess: (feeding) => {
      qc.invalidateQueries({ queryKey: feedingsKey(feeding.child_id) });
    },
  });
}

```

```

export function useStopFeeding() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (feedingId: string): Promise<Feeding> => {
      const { data, error } = await supabase
        .from('feedings')
        .update({ ended_at: new Date().toISOString() })
        .eq('id', feedingId)
        .select('*')
        .single();
      if (error) throw error;
      return data;
    },
    onSuccess: (feeding) => {
      qc.invalidateQueries({ queryKey: feedingsKey(feeding.child_id) });
    },
  });
}

```

```

export function useCreateFeeding() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (input: FeedingInsert): Promise<Feeding> => {
      const userId = await getCurrentUserId();
      const { data, error } = await supabase
        .from('feedings')
        .insert({ ...input, created_by: userId })
        .select('*')
        .single();
      if (error) throw error;
      return data;
    },
    onSuccess: (feeding) => {

```

```

        qc.invalidateQueries({ queryKey: feedingsKey(feeding.child_id) });
    },
  });
}

// app/features/auth/mutations.ts
import { useMutation } from '@tanstack/react-query';

import { supabase } from '@lib/supabase';

export function useSignIn() {
  return useMutation({
    mutationFn: async (input: { email: string; password: string }) => {
      const { error } = await supabase.auth.signInWithPassword({
        email: input.email.trim(),
        password: input.password,
      });
      if (error) throw error;
    },
  });
}

export function useSignUp() {
  return useMutation({
    mutationFn: async (input: {
      email: string;
      password: string;
      fullName: string;
    }): Promise<{ needsConfirmation: boolean }> => {
      const { error, data } = await supabase.auth.signUp({
        email: input.email.trim(),
        password: input.password,
        options: { data: { full_name: input.fullName.trim() } },
      });
      if (error) throw error;
      return { needsConfirmation: !data.session };
    },
  });
}

export function useSignOut() {
  return useMutation({
    mutationFn: async () => {
      const { error } = await supabase.auth.signOut();
      if (error) throw error;
    },
  });
}

```

```

}

export function useUpdateEmail() {
  return useMutation({
    mutationFn: async (email: string) => {
      const { error } = await supabase.auth.updateUser({ email });
      if (error) throw error;
    },
  });
}

export function useUpdatePassword() {
  return useMutation({
    mutationFn: async (password: string) => {
      const { error } = await supabase.auth.updateUser({ password });
      if (error) throw error;
    },
  });
}

export function useDeleteMyAccount() {
  return useMutation({
    mutationFn: async () => {
      const { error } = await supabase.rpc('delete_my_account');
      if (error) throw error;
      await supabase.auth.signOut();
    },
  });
}

// supabase/functions/ask-assistant/index.ts
//
// RAG pipeline:
// 1. Embed user prompt via Supabase AI (gte-small, 384 dims) - free
// 2. Semantic search over articles via pgvector cosine distance
// 3. Build context from top-3 articles
// 4. Call Groq (Llama 3.3 70B) for a grounded answer - free tier
//
// Falls back to full-text ILIKE search when embeddings are not yet
// computed (i.e., right after seeding before embed-articles runs).
//
// POST /functions/v1/ask-assistant
// { "prompt": "...", "child_age_months": 6 }

import { serve } from 'https://deno.land/std@0.224.0/http/server.ts';
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2';

```

```

const corsHeaders: Record<string, string> = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
};

const json = (status: number, body: unknown) =>
  new Response(JSON.stringify(body), {
    status,
    headers: { ...corsHeaders, 'Content-Type': 'application/json' },
  });

type ArticleSource = { id: string; title: string; category: string };

serve(async (req) => {
  if (req.method === 'OPTIONS') return new Response(null, { headers: corsHeaders });
  if (req.method !== 'POST') return json(405, { error: 'method_not_allowed' });

  const auth = req.headers.get('Authorization');
  if (!auth?.startsWith('Bearer ')) return json(401, { error: 'missing_auth' });

  let body: { prompt?: string; child_age_months?: number | null } = {};
  try {
    body = await req.json();
  } catch {
    return json(400, { error: 'invalid_json' });
  }

  const prompt = body.prompt?.trim();
  if (!prompt) return json(400, { error: 'prompt_required' });

  const childAgeMonths =
    typeof body.child_age_months === 'number' ? body.child_age_months : null;

  const supabaseUrl = Deno.env.get('SUPABASE_URL')!;
  const anonKey      = Deno.env.get('SUPABASE_ANON_KEY')!;
  const groqKey      = Deno.env.get('GROQ_API_KEY');

  if (!groqKey) return json(500, { error: 'groq_key_not_configured' });

  const client = createClient(supabaseUrl, anonKey, {
    global: { headers: { Authorization: auth } },
  });

  let articles: { id: string; title: string; body: string; category: string }[] = [];

  try {

```

					ІАЛЦ.467200.007 Д4	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		19

```

// @ts-ignore - Supabase.ai injected by edge runtime
const session = new Supabase.ai.Session('gte-small');
const output = await session.run(prompt, { mean_pool: true, normalize: true });
const embedding = Array.from(output as Float32Array);

const { data } = await client.rpc('match_articles', {
  query_embedding: embedding,
  match_count: 3,
  p_age_months: childAgeMonths,
});
articles = (data ?? []) as typeof articles;
} catch {
  // Embeddings not yet computed - fall through to text fallback.
}

if (articles.length === 0) {
  const keyword = prompt.split(/\s+/).slice(0, 4).join(' ');
  const { data } = await client
    .from('articles')
    .select('id, title, body, category')
    .or(`title.ilike.%${keyword}%,body.ilike.%${keyword}%`)
    .limit(3);
  articles = (data ?? []) as typeof articles;
}

const contextBlock = articles.length > 0
  ? articles.map((a) => `## ${a.title}\n\n${a.body}`).join('\n\n---\n\n')
  : null;

const promptLang = /[ -]/i.test(prompt) ? 'uk' : 'en';

const systemMessage = [
  'You are a helpful assistant for parents of newborns. Be concise, practical, warm.',
  childAgeMonths !== null ? `Child age: ${childAgeMonths} months.` : null,
  contextBlock
  ? `Use the following articles as context:\n\n${contextBlock}`
  : 'Answer from your general knowledge.',
  'If the question is unrelated to child health or development, politely explain.',
  promptLang === 'uk'
    ? 'LANGUAGE: Ukrainian only.'
    : 'LANGUAGE: English only.',
].filter(Boolean).join('\n\n');

const groqRes = await fetch('https://api.groq.com/openai/v1/chat/completions', {
  method: 'POST',
  headers: {

```

					ІАЛЦ.467200.007 Д4	Аркуш
						20
Зм.	Лист	№ докум.	Підп.	Дата		

```

    'Authorization': `Bearer ${groqKey}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    model: 'llama-3.3-70b-versatile',
    messages: [
      { role: 'system', content: systemMessage },
      { role: 'user', content: prompt },
    ],
    max_tokens: 600,
  }),
});

if (!groqRes.ok) {
  const detail = await groqRes.text();
  return json(502, { error: 'groq_failed', detail });
}

const groqData = await groqRes.json();
const answer: string = groqData.choices?.[0]?.message?.content ?? '';

const sources: ArticleSource[] = articles.map((a) => ({
  id: a.id,
  title: a.title,
  category: a.category,
})));

return json(200, { answer, sources });
});

```

					ІАЛЦ.467200.007 Д4	Аркуш
						21
Зм.	Лист	№ докум.	Підп.	Дата		